ARTICLE | Article in Press

Attentive temporal patch filtering for explainable streaming intrusion detection in medical IoT traffic

P. Saranya · S. Sudhakar

Received: 3 August 2026 / Accepted: 12 October 2026
© The Author(s) 2026

Abstract

Network traffic in the Internet of Medical Things (IoMT) forms an unbounded multivariate temporal sequence whose statistics shift across devices, protocols, and acquisition environments; detecting malicious segments within it is a streaming time-series classification problem constrained by bounded per-window latency, essential for temporally localized explanations, and robustness to distribution shift. This research work presents TAPX-Net, a window-local classifier that decomposes each traffic window into temporal patches, applies channel-attentive depthwise convolutional filtering, and models the patch sequence with stacked bidirectional recurrent layers before attention pooling and dual classification and reconstruction heads. Each window is classified independently, with recurrent states reset at every window boundary, making the model window-causal. A temporal attribution module (TAM) produces patch- and feature-level explanations through Integrated Gradients (IG), validated against a causal input-level ablation. A feature-harmonized adaptation stage transfers the trained model to new network environments through limited-label fine-tuning with Maximum Mean Discrepancy alignment. On the CICIoMT2024 benchmark, TAPX-Net reaches a recall of 0.9541 at an FPR of one percent and a zero-day recall of 0.8963 across ten held-out attack classes, exceeding 7 baselines on every detection metric, and under CPU-only inference on the tested configuration processes a continuous stream at 108.7 windows per second with 98.8 percent of decisions within a 10 ms per-window bound. IG tracks the causal ablation more closely than attention weights (AW) across all attack classes, and cross-dataset transfer (CDT) to the general-purpose UNSW-NB15 and ToN-IoT benchmarks recovers most of the achievable performance gain with 5,000 labeled target windows. The reconstruction head contributes a 7.46 percentage point improvement in zero-day recall, focused on the generalization regime.

Keywords Streaming time-series classification · Temporal patch decomposition · Convolutional filtering · Bounded-latency inference · XAI · Integrated gradients · Domain adaptation · Intrusion detection · IoMT

This Article in Press is shared early to give you faster access to new research. It is citable and carries a permanent DOI. The final edited version will replace it automatically.



https://doi.org/10.1038/s41598-026-76102-x

Published online: 22 October 2026

Introduction

Network traffic generated by Internet of Medical Things (IoMT) devices can be signified as a potentially unbounded multivariate sequence of packet- or flow-level feature vectors. Differences in devices, communication protocols, network workloads, and data-collection environments can alter the statistical properties of this traffic[1,2]. Detecting benign and malicious segments is therefore formulated as a streaming multivariate time-series classification problem within an IoMT intrusion-detection setting. Recent studies address real-time detection, explainability, and domain adaptation individually, but evidence remains limited for a single network that assesses all three capabilities together.

The first constraint is bounded-latency windowed inference. Detectors that depend on completed-flow statistics may defer classification until a flow terminates or times out, whereas partial-flow processing can reduce this feature-availability delay[3]. Moreover, model execution time alone does not represent detection latency because window formation and feature extraction (FE) also contribute to the interval between attack occurrence and the first correct alert[4]. A streaming detector should therefore process bounded traffic windows without waiting for session completion and report window-formation and inference delays separately.

The second constraint is attribution across the input's temporal and feature structure. Existing explainable IoMT detectors use SHAP, LIME, and related methods to identify influential traffic features[5,6]. However, these methods generally explain individual records or aggregated classes rather than jointly identifying the temporal patches and features responsible for each attack decision. The remaining problem is to produce a patch–feature explanation and test whether it faithfully represents model behaviour. Such validation should use multiple perturbation trials because the selected perturbation method can substantially impact the apparent accuracy of a time-series attribution[7]. Explainability also supports alert investigation and human oversight in healthcare environments, although an IoMT detector is not automatically classified as a high-risk system under healthcare AI regulation[8].

The third constraint is robustness to distribution shift across network environments. IoMT traffic distributions vary with device populations, protocols, attacks, FE trials, and collection conditions. Performance on a test partition from one benchmark therefore does not establish robustness on traffic collected elsewhere[2]. Recent evidence shows that domain adaptation can improve transfer between general IoT and medical IoT traffic, but only when feature compatibility and class correspondence are addressed explicitly[9]. External benchmark evaluation and clearly defined target-label budgets are consequently required.

These constraints are coupled because satisfying one while sacrificing another does not produce a deployable detector. Palaniappan and Sengan[10] introduced the MI+DL-BDA Feature Selection (FS) pipeline, which combines mutual-information filtering with deep-learning (DL)-assisted binary dragonfly optimization to find a compact IoMT traffic representation. Building on this representation, the remaining problem is a temporal detector that provides bounded-window inference, validated patch-and-feature attribution, and adaptation to external network domains using limited labelled data.

This paper proposes the Temporal Attentive Patch eXplainable Network, TAPX-Net. The model divides each sliding traffic window into temporal patches, applies channel-attentive depthwise convolutional filtering, and models the patch sequence using stacked bidirectional recurrent layers. Bidirectional processing is confined to the completed current window, and the forward and backward recurrent states are initialised to zero at the start of every window, so no recurrent state is propagated across window boundaries. TAPX-Net is therefore window-causal rather than sample-causal: a decision may depend on all observations contained in the completed current window but cannot depend on future observations or on recurrent information inherited from preceding windows.

The temporal attribution module (TAM) constructs a patch–feature attribution matrix for each attack decision using integrated gradients (IG). Attribution accuracy is measured by perturbing highly and weakly attributed regions and measuring the corresponding changes in prediction confidence. Explanation stability is evaluated across small input perturbations and independent model runs, while a model-agnostic method provides supplementary agreement analysis.

For cross-domain adaptation, TAPX-Net applies maximum mean discrepancy (MMD) alignment after mapping the source and target datasets into a compatible feature or latent representation. Alignment is followed by limited-label fine-tuning using 5,000 labeled target-domain windows; we also evaluate smaller and larger calibration sets to characterize how recovered performance varies with the labeled-data budget. In this research work, we describe the method as limited-label domain adaptation rather than few-shot learning.

The novelty of TAPX-Net lies in its integrated evaluation of bounded-window inference, joint patch–feature attribution, and limited-label cross-domain adaptation.

(a) Its first contribution is a streaming network that combines temporal patch decomposition, channel-attentive depthwise convolution, gated residual connections, window-local bidirectional recurrent processing with a zero-initialized state at every window boundary, and classification and reconstruction heads.
(b) Its second contribution is a TAM whose accuracy and stability are evaluated rather than assumed.
(c) Its third contribution is a feature-harmonised cross-dataset adaptation model with a defined target-label cost.
(d) Its fourth contribution is an evaluation using CICIoMT2024 as the primary medical-IoT benchmark and the general-purpose UNSW-NB15 and ToN-IoT datasets as external network-security domains for cross-domain transfer. Held-out-attack evaluation is separated from cross-domain transfer, and results across 5 independent runs are reported with confidence intervals, effect sizes, and paired statistical comparisons.

The remainder of this paper is organized as follows. Section "Related work" reviews IoMT intrusion detection, temporal traffic modelling, explainable detection, and domain adaptation. Section "Problem formulation" formalises the streaming detection problem. Section "Proposed model" presents TAPX-Net. Section "Experimental setup" describes the experimental setup. Section "Results and analysis" reports the results, and Section "Conclusion and future work" presents the conclusions, limitations, and future directions.

Related work

DL based intrusion detection (ID) for IoMT

Recent IoMT studies increasingly adopt hybrid neural networks. Akar et al.[11] proposed L2D2, an LSTM for multiclass detection, while Berguiga et al.[12] combined CNN-FE with LSTM sequence modelling in HIDS-IoMT. Mezina et al.[13] integrated UNet++and LSTM components to detect and classify attacks in IoMT traffic. Shaikh et al.[14] combined CNN+LSTM with reinforcement learning (RL) components, whereas Shaikh et al.[15] introduced a memory-feedback Transformer for IoMT healthcare networks.

These studies validate the value of recurrent, convolutional, and attention-based representations for IoMT-ID. However, their primary tests are limited to classification performance on benchmark data. None jointly establishes bounded-window latency, temporally localized explanations, and source-to-target adaptation in a single detector.

Temporal, attentive, and streaming detection

Several recent networks model temporal structure more explicitly. Li et al.[16] combined deformable convolution, channel and spatial attention, BiLSTM, and multi-head temporal attention. Alashjaee[17] developed an Attention-CNN-LSTM detector and reported inference latency alongside classification performance. Ghosh et al.[18] suggested TACNet, which applies multiscale convolutional filtering, LSTM processing, and temporal attention across several IoT and IIoT datasets.

Alitbi et al.[19] more directly address online traffic by incrementally encoding packets from ongoing sessions rather than waiting for completed flows. Alabbadi and Bajaber[20] combined deep classifiers with SHAP and LIME for IoT traffic described as data streams. Researchers have also studied bounded-latency processing of

multi-attribute streams outside ID. Zheng et al.[40] answer subset queries over multi-attribute data streams under strict latency constraints, and Wang et al.[41] monitor processes containing continuous and binary variables within an unsupervised network. This variable structure mirrors IoMT traffic, whose features combine continuous rate and duration measures with binary protocol-flag indicators. These works reinforce that streaming evaluation must account for full per-window processing cost, not just model execution. Much of this literature nonetheless measures computation after a sample or flow representation has already been made. Per-instance inference time should therefore not be interpreted as complete streaming latency unless it includes window attainment, preprocessing, and inference.

Explainable ID

Recent explainable IoMT detectors mainly provide feature-level post-hoc interpretations. Lipsa et al.[21] combined interpretable dimensionality reduction with random forest (RF) and SHAP on CICIoMT2024. Hafid et al.[22] used SHAP to interpret a cost-aware XGBoost detector. Alsharaiah et al.[23] integrated SHAP with a Transformer-based spoofing detector evaluated on CICIoMT2024.

Beyond IoMT-specific studies, Bin Hulayyil et al.[24] developed an explainable IoT intrusion-detection system (IDS) using laboratory and public datasets associated with Ripple20 vulnerabilities. Krishnan et al.[25] combined short- and long-term attention with LIME and SHAP for known-attack and held-out-attack detection. These works confirm that explainability is increasingly incorporated into IDS design. Their explanations remain predominantly feature-level or instance-level, and none produces a joint patch-by-feature attribution matrix or validates that matrix against an independent attribution method. Attention weights (AW), moreover, should not be treated as true explanations without separate validation.

Cross-domain and distributed robustness

Cross-environment robustness has received less integrated treatment. Alabbadi and Bajaber[26] evaluated X-FuseRLSTM across ToN-IoT, NSL-KDD, and CICIoMT2024 with residual temporal modelling and post-hoc explanation. Broadening the evaluation scope in this way is not equivalent to training on one domain and transferring to another.

Federated methods address decentralisation and privacy rather than domain adaptation directly. Kalakoti et al.[27] integrated federated DL with explainability, while Fahim-Ul-Islam et al.[28] suggested a resource-efficient federated IoMT detector. Such methods expose models to distributed data, but federated aggregation alone does not guarantee robustness to feature-space or protocol shift.

Zhang et al.[29] investigated explicit adaptation by applying domain adaptation to heterogeneous IoT-ID. Jin[30] quantified the transfer gap between Internet Protocol and Bluetooth Low Energy traffic and evaluated domain-adversarial training without target labels. These studies establish the relevance of feature harmonization and distribution alignment, yet neither combines adaptation with bounded window-level inference nor validates temporal feature attribution.

Research gap

The reviewed literature has advanced the required components separately. IoMT-specific architectures provide strong multiclass detection, temporal models capture sequential relationships, XAI methods provide feature-level explanations, and adaptation or federated-learning (FL) methods address environmental variation. Related detection and localisation problems in adjacent domains, including multi-modal ID with adaptive feature fusion in distributed services[42], fault localisation from a semantic search perspective[43], detection of faulty substructures in networks[44], and cryptographic protection tailored to medical IoT[45], address complementary aspects of secure and

reliable system operation, but none targets bounded-latency, attributable, cross-domain ID in IoMT traffic. Across the reviewed literature, no work evaluates all three central requirements in one pipeline: bounded window-level streaming latency, validated patch-and-feature attribution, and explicit source-to-target transfer. Table 1 makes this pattern explicit by naming the specific gap each study leaves open.

Table 1 Comparative analysis of recent ID literature and the remaining research gaps.

Study	Principal method	Streaming or latency evidence	Explanation type and granularity	Transfer or generalisation evidence	Remaining gap
Akar et al.[11]	L2D2 LSTM for multiclass IoMT detection	Not reported	None	Single benchmark	No streaming timing, XAI, or transfer
Berguiga et al.[12]	CNN–LSTM IoMT detector	Not reported	None	Single benchmark	Accuracy-centered offline evaluation
Mezina et al.[13]	UNet++–LSTM for IoMT traffic	Not reported	None	Single IoMT dataset	No attribution or domain transfer
Shaikh et al.[14]	CNN–LSTM with DQN and PPO	Not reported	None	Single-domain evaluation	No latency or explanation analysis
Shaikh et al.[15]	Memory-feedback Transformer	No end-to-end timing	None	Single-domain evaluation	Memory not evaluated under bounded streaming
Li et al.[16]	Deformable CNN, BiLSTM, and attention	Offline benchmark processing	Attention weights only	Three datasets evaluated separately	No validated attribution or transfer
Alashjaee[17]	Attention-CNN-LSTM	Model inference time reported	Attention weights only	Two datasets evaluated separately	Excludes window acquisition and preprocessing
Ghosh et al.[18]	Multiscale CNN–LSTM with temporal attention	Real-time suitability discussed	Temporal and channel attention	Multiple datasets evaluated separately	No end-to-end latency or attribution validation
Alitbi et al.[19]	Incremental packet encoding	Online packet processing verified	None	Novel-attack evaluation	No IoMT-specific transfer or XAI
Alabbadi and Bajaber[20]	Deep classifiers with SHAP and LIME	Dataset-based evaluation	Feature-level SHAP and LIME	Multiple datasets evaluated separately	No patch-level localisation or transfer
Lipsa et al.[21]	Interpretable reduction and RF	Not measured	Feature-level SHAP	CICIoMT2024 only	No temporal attribution or deployment timing
Hafid et al.[22]	Cost-aware XGBoost and late fusion	Not evaluated as a stream	Feature-level SHAP	Single-domain evaluation	No temporal explanation or adaptation
Alsharaiah et al.[23]	Transformer-based spoofing detector	Real-time probable discussed	Feature-level SHAP	CICIoMT2024 only	Binary spoofing focus and no transfer
Bin Hulayyil et al.[24]	ML classifiers with XAI	Not reported	Local and feature explanations	Laboratory and public datasets	No explicit adaptation or temporal explanation
Krishnan et al.[25]	Attention fusion for held-out attacks	Complexity analysed; latency not measured	LIME and SHAP	Held-out attacks within one dataset	Zero-day testing is not cross-domain transfer
Alabbadi and Bajaber[26]	Dual-path encoder, residual LSTM, and XAI	Computational efficiency discussed	Post-hoc XAI, not validated	Several datasets examined	Source-to-target calibration not established
Kalakoti et al.[27]	Federated DL with XAI	No bounded-latency assessment	Federated explanations	Distributed clients	FL is not domain adaptation
Fahim-Ul-Islam et al.[28]	Resource-efficient federated IoMT IDS	Resource efficiency assessed	None	Federated client heterogeneity	No distribution alignment or XAI
Zhang et al.[29]	Heterogeneous domain adaptation	Not evaluated for streaming	None	Explicit cross-domain adaptation	No IoMT-specific explanation or latency analysis
Jin[30]	Domain-adversarial IP-to-BLE transfer	Not assessed for streaming	None	Explicit unsupervised cross-protocol transfer	No temporal attribution or clinical-network validation
TAPX-Net (Proposed Work)	Patch-CNN-BiLSTM with dual heads	Bounded window-level latency, end-to-end	Validated patch-and-feature attribution	Limited-label source-to-target adaptation	Addresses the combined gap

Problem formulation

Streaming traffic representation

The input is a potentially unbounded sequence of packet- or flow-level observations generated by IoMT devices. To maintain consistent temporal ordering, observations are ordered within a defined stream key, such as a device, communication session, or dataset-native capture sequence, before window construction.

For one such ordered stream, the representation is expressed as in Eq. (1):

$$\mathcal{S} = \{(x_n, z_n, a_n)\}_{n=1}^{\infty}, x_n \in \mathbb{R}^d, z_n \in \{0,1\} \tag{1}$$

where x_n is the d-dimensional traffic-feature vector of observation n, z_n is its benign or malicious label available during supervised training, and a_n is its arrival timestamp.

The stream is processed through overlapping fixed-length windows to bound computation while retaining local temporal context. For window index $'k'$, let $b_k = (k-1)s + 1$ and $e_k = b_k + T - 1$ be its starting and ending observation indices.

The window is given by Eq. (2):

$$W_k = \left[x_{b_k}, x_{b_k+1}, \ldots, x_{e_k}\right] \in \mathbb{R}^{T \times d} \tag{2}$$

where T is the number of observations per window and s, with $1 \le s \le T$, is the stride. The fractional overlap between consecutive windows is expressed as in Eq. (3):

$$r_{\text{ov}} = \frac{T - s}{T} \tag{3}$$

A window is assigned a malicious label when it contains at least one malicious observation, following the rule of Eq. (4):

$$y_k = \max_{n=b_k,\ldots,e_k} z_n \tag{4}$$

This rule preserves short malicious intervals that a majority of benign observations could otherwise suppress.

For label-homogeneous windows, y_k is their common observation label.

Windowed classification objective

The streaming detection task is supervised binary classification, with one malicious-traffic probability and one hard decision assigned to every completed window.

A parameterized classifier is defined as in Eq. (5):

$$f_\theta : \mathbb{R}^{T \times d} \to [0,1], \hat{p}_k = f_\theta(W_k) \tag{5}$$

where θ are the trainable parameters and $\hat{p}_k$ is the estimated probability that W_k contains malicious traffic.

The corresponding decision follows as in Eq. (6):

$$\hat{y}_k = \mathbb{I}(\hat{p}_k \ge \tau) \tag{6}$$

where $\tau \in (0,1)$ is selected using validation data and $\mathbb{I}(\cdot)$ is the indicator function.

The classifier is window-causal. A decision for W_k can be produced only after observation x_{e_k} has arrived, and inference cannot access any observation with an index greater than e_k. The classifier may therefore use all observations within the completed current window but cannot use information from a future window.

The training objective is expressed as in Eq. (7):

$$\theta^* = \operatorname{argmin}_\theta \left[\frac{1}{|\mathcal{I}_{\text{train}}|} \sum_{k \in \mathcal{I}_{\text{train}}} \mathcal{L}_{\text{cls}}(y_k, f_\theta(W_k)) + \lambda_{\text{reg}} \mathcal{R}(\theta) \right] \tag{7}$$

where $\mathcal{I}_{\text{train}}$ is the set of training-window indices, $\mathcal{L}_{\text{cls}}$ is a binary classification loss, $\mathcal{R}(\theta)$ is a regularisation function, and $\lambda_{\text{reg}} \geq 0$ controls its contribution. Any auxiliary reconstruction or domain-alignment losses are implementation choices and are presented with the proposed framework in Section "Proposed model".

Operational constraints

The first concerns latency. Since T is several observations rather than a fixed time interval, the acquisition duration of window k is $D_k^{\text{win}} = a_{e_k} - a_{b_k}$. The conservative response-time bound measured from the first observation of window k must satisfy Eq. (8):

$$D_k^{\text{win}} + T_k^{\text{pre}} + T_k^{\text{infer}} + T_k^{\text{post}} \leq L_{\max}^{\text{det}} \tag{8}$$

where T_k^{pre} is the preprocessing time, T_k^{infer} is the classifier inference time, T_k^{post} is the decision and alert-generation time, and $L_{\max}^{\text{det}}$ does the target deployment environment permit the maximum delay. Computational latency measured after window closure is reported separately from D_k^{win}.

The second requirement concerns runtime memory and is stated as in Eq. (9):

$$M_{\text{runtime}}(f_\theta, W_k) \leq M_{\max} \tag{9}$$

where M_{runtime} includes model parameters, input buffers, maintained model states, intermediate activations, and processing workspace, and $M_{\max}$ is the memory available on the target edge gateway.

The third requirement concerns interpretability. Every positive decision must be accompanied by an attribution over the temporal and feature dimensions of the corresponding input window, produced as in Eq. (10):

$$A_k = \mathcal{E}(f_\theta, W_k) \in \mathbb{R}^{T \times d}, \hat{y}_k = 1 \tag{10}$$

where $'\mathcal{E}'$ is an attribution procedure and $A_{k,r,j}$ is the contribution assigned to feature j at temporal position $r \in \{1, \ldots, T\}$ within window $'k'$. The attribution must satisfy a predefined accuracy measure and be evaluated against an independent attribution reference.

Cross-domain generalisation requirement

Let $\mathcal{P}_s(W, y)$ and $\mathcal{P}_t(W, y)$ denote the source- and target-domain distributions after semantically harmonizing their features into a common input space. Differences in device composition, communication protocols, traffic rates, and acquisition conditions may produce the inequality of Eq. (11):

$$\mathcal{P}_s(W) \neq \mathcal{P}_t(W) \tag{11}$$

A detector trained only on samples from $\mathcal{P}_s$ must therefore be adapted using a limited labelled target set, given by Eq. (12):

$$\mathcal{D}_t^L = \{(W_i^t, y_i^t)\}_{i=1}^{N_t}, N_t \ll N_s \tag{12}$$

where N_s and N_t are the numbers of labelled source- and target-domain windows. The objective is to minimise the target-domain classification risk of Eq. (13):

$$\mathcal{R}_t(f_\theta) = \mathbb{E}_{(W,y)\sim\mathcal{P}_t}[\mathcal{L}_{\text{cls}}(y, f_\theta(W))] \tag{13}$$

while satisfying the latency, memory, and attribution requirements of Eqs. (8)–(10).

Proposed model

Overview

The proposed model comprises five stages, as exemplified in Fig. 1: flow construction, preprocessing, offline FS, streaming classification, and post-decision attribution. Packet captures are first converted into statistical flow records; datasets supplied as flow tables enter directly at preprocessing. Invalid values are removed, missing values are imputed, and records are ordered within their defined device, session, or capture sequence. Imputation and z-score normalisation parameters are estimated exclusively from the training partition and subsequently frozen. The MI+DL-BDA method of Palaniappan and Sengan[10] then selects a compact subset of $'d'$ features using only the training partition.

TAPX-Net operates as a strictly window-local streaming classifier. Incoming traffic observations are processed sequentially into overlapping fixed-length windows, but each completed window is evaluated independently by the neural network (NN). Within a window, the stacked BiLSTM layers process the patch sequence bidirectionally so each patch representation can incorporate context from all patches in the current window. The forward and backward hidden and cell states are initialised to zero at the start of every window, and no recurrent state is propagated across window boundaries.

During online operation, the frozen feature mask is applied to incoming records, and the resulting stream is divided into overlapping windows. Each completed window is decomposed into temporal patches and processed by TAPX-Net to obtain the per-window malicious-traffic probability $\hat{p}_k$ and the binary decision $\hat{y}_k$. A positive decision triggers an immediate detection alert. The TAM subsequently produces a patch–feature attribution matrix and appends the highest model-attributed temporal regions and features to the alert record.

Feature-mask optimisation is performed offline and therefore falls outside the response-time bound in Eq. (8). Applying the frozen mask, normalising incoming records, and constructing the current window remain online operations and are included in T_k^{pre}. Classification requires one forward inference pass. Because each completed window is processed independently, with forward and backward recurrent states initialised to zero at the window

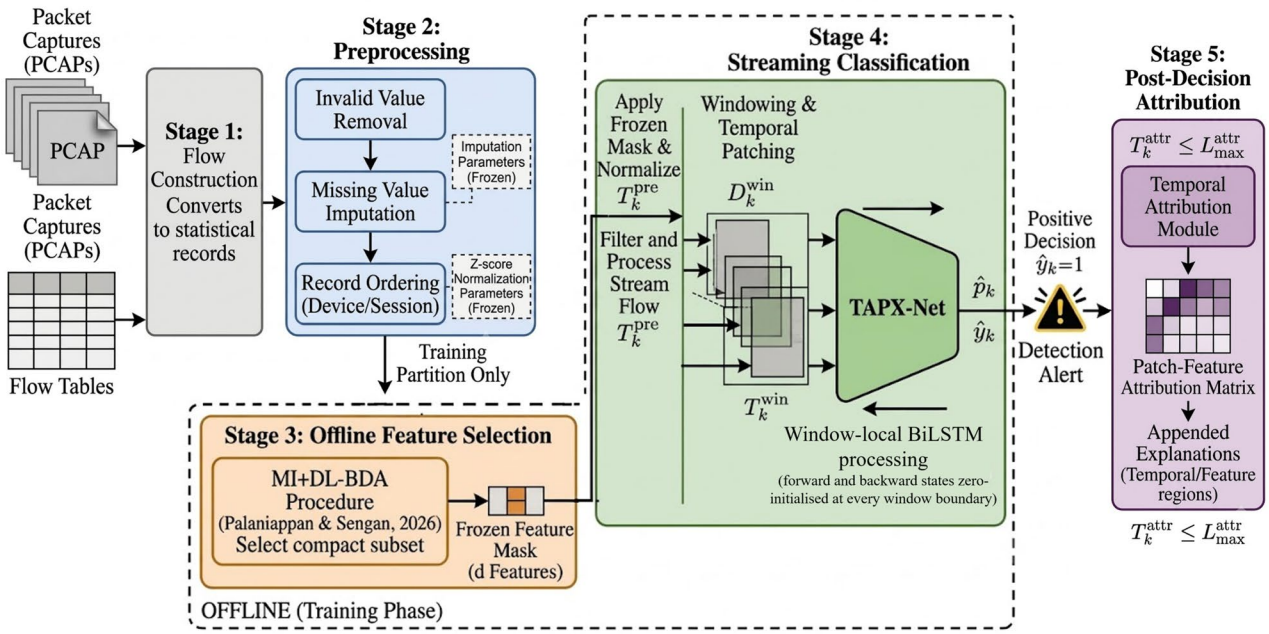


Fig. 1 Five-stage pipeline of the proposed TAPX-Net

boundary, the online forward computation is identical to that used in training, and no recurrent state is carried across consecutive windows.

The detection delay comprises the window-acquisition duration D_k^{win}, preprocessing time T_k^{pre}, inference time T_k^{infer}, and decision time T_k^{post}. Attribution is executed only after a positive decision and does not delay the initial alarm. Its runtime T_k^{attr} is reported separately against the explanation-time limit $L_{\max}^{\text{attr}}$.

Data preprocessing pipeline

Four operations prepare the flow-feature matrix for window construction.

(a) Non-finite numerical values are treated as missing and replaced with the corresponding training-partition median. Features that remain undefined or have zero variance in the training partition are dropped.
(b) Each retained numerical feature is standardised using the training-partition mean and standard deviation, producing zero-centered features on a comparable scale.
(c) This study retains stream identifiers, timestamps, and class labels as metadata and does not standardize them. We freeze all imputation and scaling parameters and apply them to validation and test records.
(d) In cross-domain experiments, this work computes any target-specific preprocessing parameters only from the permitted target calibration partition, never from the target test partition.

This work establishes data partitions at the stream-key or capture-session level before window construction, ensuring related observations and overlapping windows do not appear in different partitions. Within each partition, records belonging to the same stream key are ordered by flow-start timestamp and segmented into windows of length $'T'$ and stride s according to Eq. (2). Window construction restarts at every stream-key, session, or partition boundary, so no window combines independent sequences or crosses a data-split boundary. Each completed window is labelled according to Eq. (4): $y_k = 1$ when at least one constituent observation is malicious and $y_k = 0$ otherwise.

Feature selection: MI + DL-BDA

In this study, we adopt the MI + DL-BDA of Palaniappan and Sengan[10] as the FS stage and summarise it here to establish the notation and the selected subset the classifier receives. It operates in two stages: a mutual-information (MI) filter that removes irrelevant and redundant features, followed by a binary metaheuristic search whose fitness is tested by a frozen neural surrogate. In Algorithm 1, $\mathcal{I}_{train}$ and $\mathcal{I}_{val}$ denote the training and validation index sets of the chronological 60:20:20 partition defined in Section "Data preprocessing pipeline". All FS statistics are estimated on $\mathcal{I}_{train}$ and candidate masks are scored on $\mathcal{I}_{val}$.

MI filtering

Statistical dependence between each feature X_f and the binary label Y is estimated on the training partition using a k-nearest-neighbour (k-NN)-MI estimator, which requires no density model or discretisation. Features are ranked in descending order of $R(f) = \text{MI}(X_f; Y)$, and the top $\lfloor \rho_{\text{ret}} D \rfloor$ are retained under the retention ratio ρ_{ret}, where D is the cleaned feature count. Pairwise Pearson correlations are then computed over the retained set, and for any pair exceeding the pruning threshold ρ_{thresh} the lower-ranked feature is removed. The retention ratio was $\rho_{ret} = 0.55$, the k-NN estimator used $k = 5$ neighbours, and the correlation pruning threshold was $\rho_{\text{thresh}} = 0.90$. The output is a reduced matrix $X' \in \mathbb{R}^{N \times F'}$ of mutually informative, non-redundant features.

DL-BDA search with a ProxyNet surrogate

Conventional wrapper selection retrains a classifier for every candidate subset, which is computationally intractable over the search space of F' features. The DL-BDA stage removes this cost by delegating fitness evaluation to a frozen surrogate, ProxyNet, trained once on the training partition.

ProxyNet is a multi-layer perceptron with an efficient channel attention module (ECAM) that recalibrates channel responses through local cross-channel interaction (Fig. 2). Its two hidden layers have widths of 256 and 128; the ECAM uses a kernel size of 9, and dropout is set to 0.30. It is trained once with class-weighted binary cross-entropy, with weights set inversely to class frequency to counter the severe imbalance of IoMT traffic; its weights are then frozen and used only as an inference oracle during the search.

The binary dragonfly algorithm (BDA) maintains a population of agents, each a binary feature mask $m \in \{0,1\}^{F'}$. Agent positions evolve through separation, alignment, cohesion, attraction to the best solution, and distraction from the worst, combined in a velocity update and mapped to binary decisions through an S-shaped transfer function with velocity clamping. For each candidate mask, unselected features are zeroed in the validation partition; ProxyNet produces detection-probability estimates without retraining, and the safety-weighted fitness is

$$J(m) = \alpha R_{\mathrm{FPR}\le 1\%}(m) + \beta\,\mathrm{PR-AUC}(m) - \gamma \frac{\|m\|_0}{F'} - \delta \frac{T(m)}{T_{\max}} \tag{14}$$

where $R_{FPR\le 1\%}$ is the recall at a false-positive rate (FPR) of at most 1%, $PR-AUC$ is the area under the precision-recall curve, $\|m\|_0/F'$ is the normalised subset cardinality, $T(m) = T_0 + c\|m\|_0$ is an estimated inference cost under a linear model with fixed overhead T_0 and per-feature coefficient c, T_{max} is the maximum admissible cost, and the non-negative weights satisfy $\alpha + \beta + \gamma + \delta = 1$. The recall term addresses the safety–critical detection requirement and the cost term enforces deployment feasibility, so the selected subset reflects both objectives rather than accuracy alone.

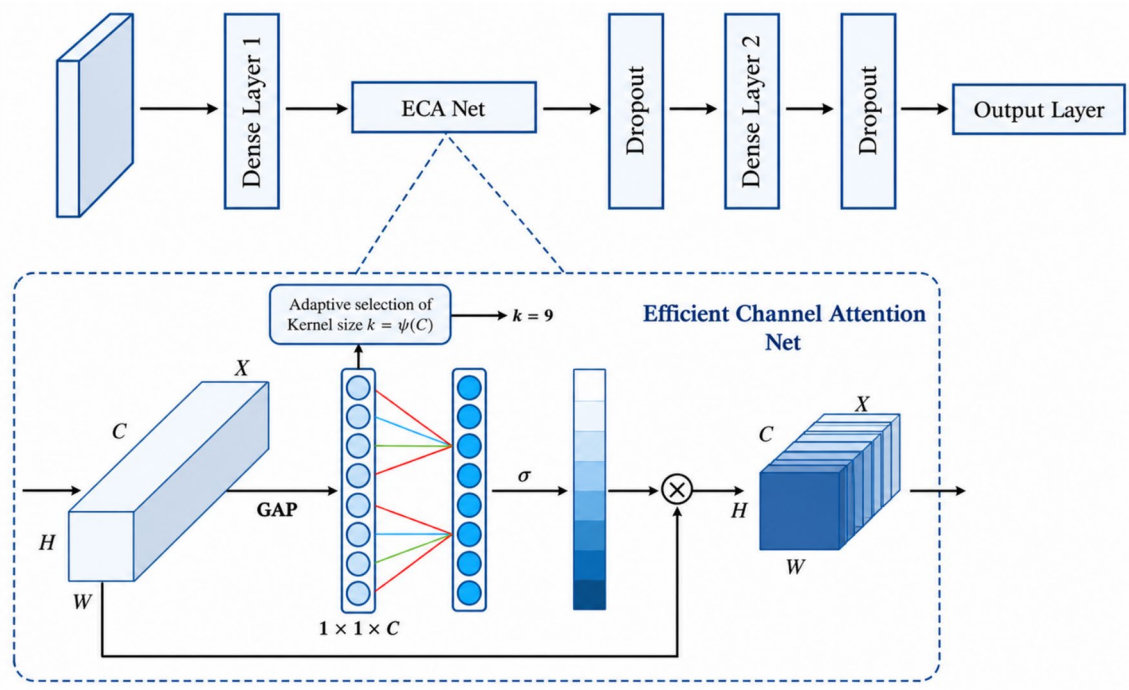


Fig. 2 ProxyNet

The BDA used a population of 40 agents for at most 20 iterations, with early termination after 5 consecutive iterations without an improvement greater than 10^{-3}. Candidate velocities were clamped and converted to binary masks through the logistic S-shaped transfer function, with the inertia weight decayed linearly from 0.9 to 0.4. The fitness weights were $\alpha = 0.50$, $\beta = 0.30$, $\gamma = 0.10$, and $\delta = 0.10$. The cost model used $T_0 = 7.8$ ms, $c = 0.09\,ms$ per retained feature, and $T_{max} = 10.0\,ms$.

The search stops at a fixed iteration cost or on stagnation, and the converged mask m^* is the selected subset of cardinality $d = \|m^*\|_0$. The corresponding columns of the ordered matrix $\hat{X}_\Pi$ constitute the input stream S of Eq. (1), and $'d'$ carries through all downstream equations. Algorithm 1 states the full procedure.

Input: Training matrix X̂_Π ∈ ℝ^(N×D), labels y ∈ {0,1}^N
Output: Optimal mask m* ∈ {0,1}^(F'), selected dimension d = ‖m*‖₀
Stage 1 — MI Filtering
For Each feature f = 1 to D:
  R(f) ← MI(X_f ; Y) via k-NN estimator over I_train
Rank features by R(f) descending
Retain top F' = ⌊ρ_ret · D⌋ features
For Each retained pair (f_i, f_j) with R(f_i) > R(f_j):
  If |ρ(f_i, f_j)| > ρ_thresh: remove f_j
Output reduced matrix X' ∈ ℝ^(N×F')
Stage 2 — ProxyNet Surrogate
Train ProxyNet (MLP + ECA) on X'[I_train] with class-weighted BCE
Freeze ProxyNet → inference oracle
Stage 3 — BDA Search
Initialise population of masks {m_i} ∈ {0,1}^(F')
For t = 1 to T_max:
  For Each agent i:
    Zero unselected features in X'[I_val] via m_i
    Estimate R_FPR≤1%, PR-AUC through frozen ProxyNet
    Evaluate J(m_i) by Eq. (14)
    Update velocity (separation, alignment, cohesion,
      Food attraction, enemy distraction)
    Apply S-shaped transfer → binary mask; clamp velocity
  Update global best m⁺ and worst m⁻ by J(·)
  Decay inertia weight linearly
  If stagnation ≥ patience: break
Return m* = m⁺, d = ‖m*‖₀

Algorithm 1 MI+DL—BDA-Based FS.

TAPX-Net

TAPX-Net answers the three constraints of Section "Problem formulation", namely bounded-latency windowed inference, temporal-feature attribution (TFA), and robustness under distribution shift, through 4 components applied in sequence: patch embedding, channel-attentive convolutional filtering, stacked bidirectional recurrence, and attention pooling feeding dual output heads. The encoder is trained on the source domain and supports the domain-adaptation stage; it does not by itself resolve distribution shift (Fig. 3).

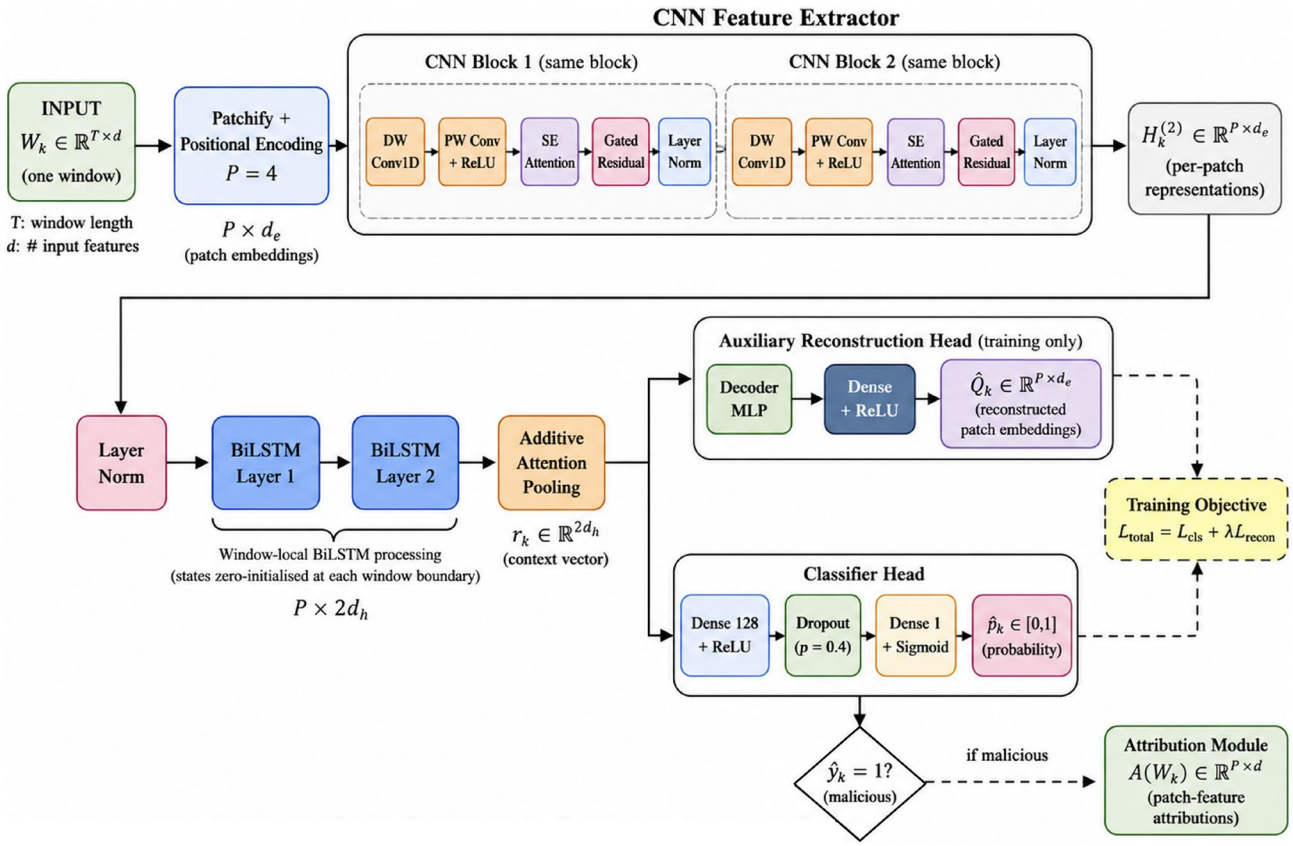


Fig. 3 TAPX-Net

Patchification and positional embedding

This study imposes structured temporal segmentation on each window before any learned transformation, giving the downstream stages a bounded temporal unit. Each window $W_k \in \mathbb{R}^{T\times d}$ is divided into $'P'$ contiguous non-overlapping patches, each patch $p_{k,i} \in \mathbb{R}^{(T/P)\times d}$ spanning a fixed temporal segment of length $L = T/P$ with non-learnable boundaries. Because the window length is fixed at $T = 32$ observations and the patches are contiguous, non-overlapping, and equal in length, the patch length and patch count are related by $L = T/P$, so the admissible patch counts are the divisors of $'T'$. The patch count was treated as a tunable hyperparameter and tested over $P \in \{2,4,8,16\}$, corresponding respectively to patch lengths of 16, 8, 4, and 2 observations; larger values were not measured because they reduce each patch toward a single observation and remove the local temporal aggregation that the patch representation is intended to provide.

Each patch is flattened and mapped into a shared embedding space through a trainable affine projection with an additive learnable positional encoding, as defined in Eq. (15):

$$q_{k,i} = \text{Linear}\left(\text{Flatten}\left(p_{k,i}\right)\right) + \rho_i, q_{k,i} \in \mathbb{R}^{d_e} \tag{15}$$

where $\text{Linear} : \mathbb{R}^{L\cdot d} \to \mathbb{R}^{d_e}$ is the projection, $\rho_i \in \mathbb{R}^{d_e}$ is the positional encoding for patch index , and d_e is the embedding dimension used throughout the subsequent layers. The projection is applied identically to each patch, producing the ordered sequence $\{q_{k,1}, \ldots, q_{k,P}\}$ that enters the convolutional stage.

The recurrent layers that follow are order-sensitive, so the model already has access to absolute patch position through the sequence itself. The positional encoding is an optional addition that improves absolute-position awareness in the convolutional stage, whose temporal kernels are translation-equivariant and therefore insensitive

to where a pattern occurs within the window. Its trainable parameters are the projection weight and bias, of size $L \cdot d \cdot d_e + d_e$, and the positional encodings, of size $P \cdot d_e$.

Convolutional blocks with channel attention and gated residuals

Two cascaded convolutional blocks transform the patch-embedding sequence into a channel-recalibrated, residual-regulated feature sequence reflecting local cross-patch interaction and per-channel discriminative importance (Fig. 4).

A. *Depthwise Separable Convolution.* A full convolution over the d_e-dimensional sequence couples spatial filtering and channel mixing, at a cost scaling with the product of input and output channel counts. Depthwise separable convolution decouples them, applying an independent temporal kernel per channel before recombining channels through a pointwise projection, as defined in Eq. (16):

$$H_{\text{pw}} = \text{Conv}_{1\times1}\left(\oplus_{c=1}^{d_e}(H_{\text{in}}[:,c] * k_c)\right) \tag{16}$$

where k_c is the depthwise kernel for channel c, $*$ is convolution along the patch dimension, $\oplus$ is channel-wise concatenation, and $\text{Conv}_{1\times1}$ recombines channels into $H_{\text{pw}} \in \mathbb{R}^{P\times d_e}$. For kernel size $'k'$, this reduces the parameter count from $'k' \cdot d_e^2$ for a full convolution to $k \cdot d_e + d_e^2$, a reduction by a factor of approximately $1/d_e + 1/k$ that supports the memory constraint of Eq. (9).

B. *Squeeze-and-Excitation Channel Attention.* The pointwise output weights all d_e channels equally, but not all channels carry equal discriminative signal for every attack type. A global descriptor is formed per channel by averaging over patch positions, as defined in Eq. (17):

$$\mu_c = \frac{1}{P}\sum_{i=1}^{P} H_{\text{pw}}[i,c] \tag{17}$$

The descriptor is passed through a two-layer bottleneck with ReLU and sigmoid to produce a channel-scaling vector, as defined in Eq. (18):

$$\beta = \sigma(W_2\,\text{ReLU}(W_1\mu)) \tag{18}$$

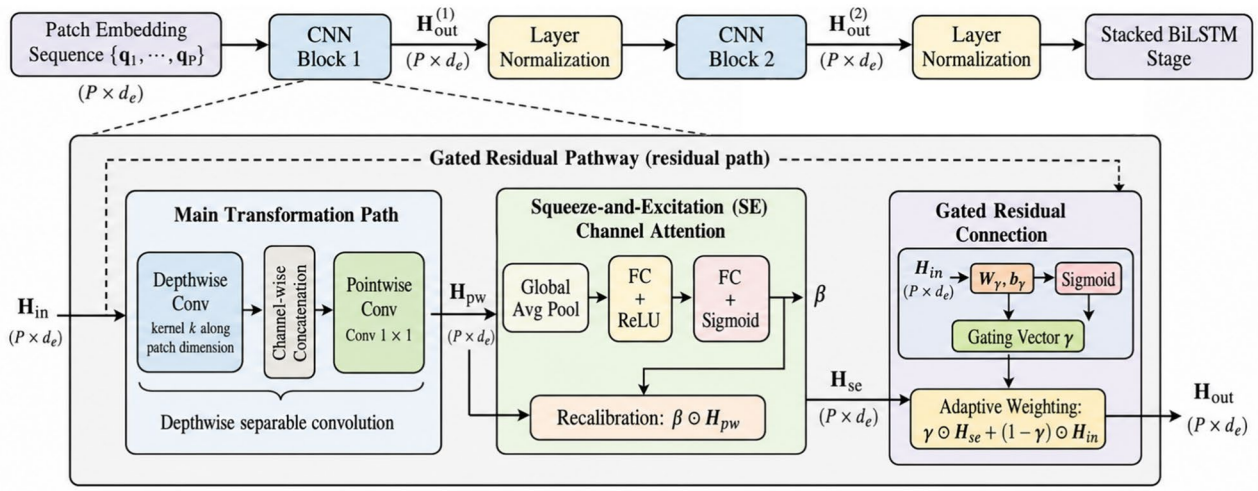


Fig. 4 Convolutional Block Structure

where $W_1 \in \mathbb{R}^{(d_e/r)\times d_e}$ compresses to a bottleneck of width d_e/r, $W_2 \in \mathbb{R}^{d_e\times(d_e/r)}$ expands back, $'r'$ is the reduction ratio, and $\beta \in [0,1]^{d_e}$. The recalibrated map is $H_{se} = \beta \odot H_{pw}$, with $\odot$ broadcast across the patch dimension. Suppressing low-relevance channels here sharpens the embedding structure that the recurrent layers subsequently model.

C. *Gated Residual Connection.* A fixed additive shortcut would propagate the full block input, including low-relevance channels, unchanged into the recurrent and attention stages. A learned channel-wise gate interpolates instead between the attended output and the block input, as defined in Eq. (19):

$$H_{out} = \gamma \odot H_{se} + (1-\gamma) \odot H_{in} \tag{19}$$

where $\gamma = \sigma\left(W_\gamma \overline{H}_{in} + b_\gamma\right) \in \mathbb{R}^{d_e}$ is a channel-wise gate computed from the patch-averaged block input $\overline{H}_{in} = \frac{1}{P}\sum_{i=1}^{P} H_{in}[i,:]$ and broadcast across the patch dimension. The gate weights the attended output highly where the convolution has produced discriminative signal and weights the residual path highly where the input already carries sufficient structure. Layer normalisation is applied to H_{out}. Two such blocks are cascaded, the normalised output of the first serving as H_{in} for the second, and the second block output $H_{out}^{(2)} \in \mathbb{R}^{P\times d_e}$ enters the recurrent stage.

Stacked BiLSTM layers

The convolutional output captures local cross-patch interaction but carries no explicit model of sequential dependency across patch order; two stacked bidirectional recurrent layers supply that dependency, computed entirely within each window. Each window is processed independently, and no recurrent state is carried across window boundaries, so the model is window-local and trivially window-causal. For each layer $l \in \{1,2\}$, forward and backward cells process the patch sequence in opposite directions. The forward state at patch $'i'$ is defined as in Eq. (20):

$$\vec{h}_i^{(l)} = \overrightarrow{\mathrm{LSTM}}^{(l)}\left(x_i^{(l-1)}, \vec{h}_{i-1}^{(l)}\right) \tag{20}$$

and the backward state is defined as in Eq. (21):

$$\overleftarrow{h}_i^{(l)} = \overleftarrow{\mathrm{LSTM}}^{(l)}\left(x_i^{(l-1)}, \overleftarrow{h}_{i+1}^{(l)}\right) \tag{21}$$

where $x_i^{(l-1)}$ is the input at patch $'i'$ from the preceding layer, equal to $H_{out}^{(2)}[i,:]$ for $l=1$ and to the concatenated layer-1 output for $l=2$, with both directional states initialised to '0' at each window boundary.

The two directions are concatenated, as defined in Eq. (22):

$$b_i^{(l)} = \left[\vec{h}_i^{(l)};\ \overleftarrow{h}_i^{(l)}\right] \tag{22}$$

with each direction of hidden width d_h, so $b_i^{(l)} \in \mathbb{R}^{2d_h}$. The stacked output is the sequence $\left\{b_1^{(2)}, \ldots, b_P^{(2)}\right\}$, each element encoding the bidirectional context of its patch conditioned on all other patches in the window.

Additive attention pooling

The stacked recurrent output is a sequence of P patch representations of equal dimensionality, which additive attention pooling collapses into a single window representation by weighting each patch according to a learned relevance score. Each $b_i^{(2)}$ is first projected into an attention subspace, as defined in Eq. (23):

$$\phi_i = \tanh\left(W_a b_i^{(2)} + b_a\right) \tag{23}$$

where $W_a \in \mathbb{R}^{d_a \times 2d_h}$ projects to the attention dimension d_a and $b_a \in \mathbb{R}^{d_a}$ is the bias. The attention weight for patch $'i'$ is obtained by scoring ϕ_i against a learnable context vector $v_a \in \mathbb{R}^{d_a}$ under SoftMax normalisation, as defined in Eq. (24):

$$\omega_i = \frac{\exp\left(\phi_i^\top v_a\right)}{\sum_{j=1}^{P} \exp\left(\phi_j^\top v_a\right)} \tag{24}$$

so that $\sum_{i=1}^{P} \omega_i = 1$. The window representation is the weighted sum, as defined in Eq. (25):

$$r = \sum_{i=1}^{P} \omega_i \, b_i^{(2)}, r \in \mathbb{R}^{2d_h} \tag{25}$$

which serves as the window-level context vector entering the output heads. The weights $\{\omega_1, \ldots, \omega_P\}$ act as an internal weighting signal that determines each patch's contribution to $'r'$ and thereby to the decision. They are not treated as an explanation on their own, since attention magnitude does not establish causal importance[31]; their use in attribution is restricted to the supplementary agreement check of Section "TAM", gated by the accuracy evaluation defined there.

Dual output heads

The context vector $'r'$ is routed to two parallel heads whose joint objective addresses both the classification requirement of Eq. (7) and the generalisation demand imposed by zero-day traffic. The classification head produces the per-window decision, and the reconstruction head regularises the representation through an auxiliary self-supervised task.

A. *Classification head* The context vector is mapped through a two-layer network with dropout. The hidden representation is defined as in Eq. (26):

$$o_{\text{cls}} = \text{ReLU}\left(W_{\text{cls1}} r + b_{\text{cls1}}\right) \tag{26}$$

and the detection probability is obtained from the dropout-regularised hidden representation, as defined in Eq. (27):

$$\hat{p}_k = \sigma(z_k), z_k = W_{\text{cls2}} \, \text{Dropout}\left(o_{\text{cls}}\right) + b_{\text{cls2}} \tag{27}$$

where $W_{\text{cls1}} \in \mathbb{R}^{d_c \times 2d_h}$ projects to the intermediate dimension d_c, $W_{\text{cls2}} \in \mathbb{R}^{1 \times d_c}$, z_k is the malicious-class logit, $'\sigma'$ is the sigmoid producing $\hat{p}_k \in [0,1]$, and dropout of rate $'\zeta'$ is applied during training. The hard label $\hat{y}_k$ follows by thresholding per Eq. (6).

B. *Reconstruction Head* Zero-day traffic produces flow patterns that share low-level temporal structure with known attack types, such as inter-arrival regularity and directional byte asymmetry, even when their high-level categorical signatures are absent from the training distribution. A classifier trained on the classification loss alone compresses the representation toward the discriminative subspace of the training taxonomy and discards structure that does not separate known classes. The reconstruction head counteracts this by reconstructing the patch-embedding sequence from the recurrent representation, so the joint objective forces the encoder to retain a broader representational basis.

For each patch $'i'$, the first decoder layer is defined as in Eq. (28):

$$o_{\text{rec1},i} = \text{ReLU}\left(W_{\text{rec1}} b_i^{(2)} + b_{\text{rec1}}\right) \tag{28}$$

The second decoder layer is defined as in Eq. (29):

$$o_{\text{rec2},i} = \text{ReLU}\left(W_{\text{rec2}} o_{\text{rec1},i} + b_{\text{rec2}}\right) \tag{29}$$

and the reconstructed embedding is produced by the output layer, as defined in Eq. (30):

$$\hat{q}_{k,i} = W_{\text{rec3}} o_{\text{rec2},i} + b_{\text{rec3}} \tag{30}$$

where $W_{\text{rec1}} \in \mathbb{R}^{d_{r1} \times 2d_h}$, $W_{\text{rec2}} \in \mathbb{R}^{d_{r2} \times d_{r1}}$, and $W_{\text{rec3}} \in \mathbb{R}^{d_e \times d_{r2}}$ projects to the embedding dimension, giving $\hat{q}_{k,i} \in \mathbb{R}^{d_e}$. The target of reconstruction is the input patch embedding $q_{k,i}$ of Eq. (15); reconstructing the input embeddings rather than the pooled vector avoids a trivial identity mapping, since the target differs from the decoder input. The reconstruction loss is defined in Section "Training objective and optimisation". The reconstruction head is active only during training; at inference, only the classification head runs.

TAM

The TAM runs only after a positive decision and constructs a temporal-feature attribution for the window. IG is the primary attribution method, and the retained attention weights are used only for a supplementary agreement check. The module adds no trainable parameters and does not alter the decision. Its runtime is measured separately as T_k^{attr} against the explanation-time cost $L_{\max}^{\text{attr}}$, and is not part of the detection latency.

IG attribution IG satisfies the completeness and sensitivity axioms that AW cannot guarantee[32], attributing the malicious-class logit to individual features at each temporal position by accumulating gradients along a straight-line path from a baseline to the window input. Attributing the logit z_k of Eq. (27) rather than the sigmoid probability avoids gradient saturation near the decision extremes. For temporal position $'r'$ and feature $'j'$, over $'M'$ steps, the attribution is defined as in Eq. (31):

$$\text{IG}_{k,r,j} \approx \left(x_{r,j} - x_{r,j}'\right) \times \frac{1}{M} \sum_{m=1}^{M} \frac{\partial z_k \left(x' + \frac{m}{M}\left(x - x'\right)\right)}{\partial x_{r,j}} \tag{31}$$

where $x_{r,j}$ is the input value at position $'r'$, feature $'j'$, and $x_{r,j}' = 0$ is the baseline. The approximation requires $'M'$ gradient evaluations along the path rather than a single pass. The result is the full-resolution attribution $A_k^T \in \mathbb{R}^{T \times d}$ required by Eq. (10). TAPX-Net additionally aggregates this within patches to a patch-level matrix, as defined in Eq. (32):

$$A_{k,i,j} = \sum_{r=(i-1)L+1}^{iL} \text{IG}_{k,r,j}, A\left(W_k\right) \in \mathbb{R}^{P \times d} \tag{32}$$

where $L = T/P$ is the patch length and $A_{k,i,j}$ is the contribution of feature $'j'$ within patch $'i'$. By the completeness axiom, the elements of A_k^T sum to $z_k - z_k^0$, where z_k^0 is the logit at the baseline.

Attention agreement check The retained attention weights are compared against the gradient-based attribution to assess whether the two rankings agree, without treating either as ground truth. Patch importance is first localised by an input-level ablation: for each patch $'i'$, the corresponding input positions are replaced by the baseline, and the full model is rerun to obtain the ablated prediction $\hat{p}_k^{(\neg i)}$, giving the causal drop $\delta_i = \hat{p}_k - \hat{p}_k^{(\neg i)}$. Because the model is window-local, this rerun removes patch $'i'$ cleanly, with no inherited state to hold fixed. The patch-marginal gra-

dient score $\overline{A}_i = \sum_{j=1}^{d} | A_{k,i,j} |$, the ablation drop δ_i, and the AW as ω_i are compared by rank correlation; agreement supports the joint use of attention and gradients, and disagreement flags the AW as unreliable for that window.

Per-class attribution profiling Per-class profiles are constructed from correctly classified training and validation windows, frozen, and their stability assessed on the test set, so that no test data enters profile construction.

For class $'c'$, the signed profile aggregates the patch-level attribution across the profiling set, as defined in Eq. (33):

$$\overline{A}^{(c)} = \frac{1}{| \mathcal{I}_c |} \sum_{k \in \mathcal{I}_c} A(W_k) \tag{33}$$

and the mean absolute profile is computed in parallel, as defined in Eq. (34):

$$\overline{A}^{(c)}_{\text{abs}} = \frac{1}{| \mathcal{I}_c |} \sum_{k \in \mathcal{I}_c} | A(W_k) | \tag{34}$$

where $\mathcal{I}_c$ is the index set of correctly classified profiling windows of class $'c'$. The signed profile of Eq. (33) preserves whether each feature supports or opposes the malicious prediction, while the absolute profile of Eq. (34) measures overall attribution magnitude without cancellation. The reported profile involves the dominant patch position and the top-ranked features per class, and the results report its rank stability between the profiling set and the test set. The profile provides time-localised, feature-level signatures for each known attack type and a reference against which borderline detections near the threshold $'\tau'$ of Eq. (6) are compared during human-in-the-loop escalation review.

Training objective and optimisation

The composite objective combines a focal classification loss with the reconstruction regulariser of Section "Dual output heads", optimised under a standard adaptive schedule. Each loss term addresses a constraint specific to IoMT-ID: class imbalance for the classifier and representational breadth for generalisation to unseen traffic.

A. *Focal Loss for Classification* Binary cross-entropy weights all samples equally, so the majority attack class dominates the gradient and the benign minority is underweighted, which degrades recall at the low FPR required for deployment. Focal loss down-weights confidently classified samples and concentrates the gradient on hard and misclassified instances. For window $'k'$, the per-sample loss is defined as in Eq. (35):

$$\mathcal{L}^{(k)}_{\text{focal}} = -\eta_{y_k} \left(1 - \hat{p}_{y_k}\right)^{\varphi} \text{Log}\left(\hat{p}_{y_k}\right) \tag{35}$$

where $\hat{p}_{y_k} = y_k \hat{p}_k + (1 - y_k)(1 - \hat{p}_k)$ is the probability assigned to the ground-truth class, y_k is the window label of Eq. (4), φ is the focusing parameter, and η_{y_k} is the class weight. The class weight is set by inverse frequency on the training partition, as defined in Eq. (36):

$$\eta_y = \frac{| \mathcal{I}_{\text{train}} |}{| \mathcal{I}^{(y)}_{\text{train}} |} \tag{36}$$

where $| \mathcal{I}^{(y)}_{\text{train}} |$ is the number of training windows labelled $'y'$, raising the gradient contribution of the benign minority. The aggregate loss is the mean over the training partition, as defined in Eq. (37):

$$\mathcal{L}_{\text{focal}} = \frac{1}{| \mathcal{I}_{\text{train}} |} \sum_{k \in \mathcal{I}_{\text{train}}} \mathcal{L}^{(k)}_{\text{focal}} \tag{37}$$

B. *Reconstruction Regularisation* The reconstruction head penalises the deviation of each reconstructed patch embedding $\hat{q}_{k,i}$ of Eq. (30) from its target $q_{k,i}$ of Eq. (15), averaged over patches and over a mini-batch of size B, as defined in Eq. (38):

$$\mathcal{L}_{\text{rec}} = \frac{1}{BP}\sum_{k=1}^{B}\sum_{i=1}^{P} \| q_{k,i} - \hat{q}_{k,i} \|_2^2 \tag{38}$$

C. *Composite Objective* The total loss combines the two terms through a fixed weight, as defined in Eq. (39):

$$\mathcal{L}_{\text{total}} = \mathcal{L}_{\text{focal}} + \lambda_{\text{rec}}\mathcal{L}_{\text{rec}} \tag{39}$$

where λ_{rec} controls the reconstruction contribution; a small value maintains classification dominance while retaining enough gradient through the reconstruction path to enforce representational breadth. The class weights and λ_{rec} are fixed from the training partition before reading any validation or test data.

D. *Optimisation* The objective is minimised with the Adam optimiser under a cosine-annealing schedule with linear warmup, decaying the learning rate from η_0 to $\eta_{\min}$, as defined in Eq. (40):

$$\eta_e = \eta_{\min} + \frac{1}{2}(\eta_0 - \eta_{\min})\left(1 + \text{Cos}\frac{\pi\,(e - e_w)}{E_{\max} - e_w}\right) \tag{40}$$

where e is the epoch, e_w the warmup duration, and $E_{\max}$ the maximum epoch count. Gradients are clipped to a maximum ℓ_2 norm ν at each step. This study selected hyperparameters on the training partition through temporally blocked validation over candidate window lengths, overlap ratios, hidden dimensions, learning rates, and batch sizes, ranked by recall at an FPR of at most one percent, then by PR-AUC, with latency and memory as deployment constraints. The selected configuration is reported in Table 2.

All five runs used the same fixed chronological train, validation, and test partitions; partition membership was not resampled across seeds. The seed affected only parameter initialization, mini-batch shuffling, dropout, and stochastic FS initialization.

Cross-dataset domain adaptation

A classifier trained on one traffic benchmark carries an implicit hypothesis that the feature statistics, class proportions, and temporal patterns of the training environment persist at deployment, which IoMT set-up across hospitals, manufacturers, and protocol conventions violates. This stage addresses the resulting gap through feature-space alignment followed by limited-label fine-tuning, realising the transfer objective of Section "Cross-domain generalisation requirement".

Feature-space alignment

The d features selected by MI+DL-BDA from the source dataset $\mathcal{D}_s$ are not guaranteed to be present in a target dataset $\mathcal{D}_t$ captured under different device configurations or flow-generation tools, so an explicit alignment precedes fine-tuning. Let $\mathcal{F}_s$ is the selected source feature set and $\mathcal{F}_t$ the full target feature set. Their overlap is quantified by the Jaccard coefficient, as defined in Eq. (41):

$$J(\mathcal{F}_s, \mathcal{F}_t) = \frac{|\,\mathcal{F}_s \cap \mathcal{F}_t\,|}{|\,\mathcal{F}_s \cup \mathcal{F}_t\,|} \tag{41}$$

Table 2 TAPX-Net Hyperparameter Configuration.

Component	Hyperparameter	Symbol	Value
Patchification	Window length	T	32
	Stride	s	16
	Number of patches	P	4
	Embedding dimension	d_e	256
CNN blocks	Number of blocks	—	2
	Kernel size	k	3
	SE reduction ratio	r	16
BiLSTM	Number of layers		2
	Hidden dimension per direction	d_h	256
	Concatenated output dimension	$2d_h$	512
Attention Pooling	Attention dimension	d_a	128
Classification Head	Intermediate dimension	d_c	128
	Dropout rate	ζ	0.4
Reconstruction Head	First intermediate dimension	d_{r1}	256
	Second intermediate dimension	d_{r2}	128
	Output dimension (per patch)	d_e	256
Training	Optimiser	—	Adam
	Initial learning rate	η_0	10^{-3}
	Minimum learning rate	$\eta_{\min}$	10^{-6}
	Schedule	—	Cosine annealing
	Warmup duration	e_w	10 epochs
	Maximum epochs	$E_{\max}$	50
	Batch size	B	32
	Gradient-clip threshold	ν	1.0
	Early-stopping patience	—	10 epochs
Loss	Focal focusing parameter	φ	2.0
	Reconstruction weight	λ_{rec}	0.1
	Class weighting	—	Inverse frequency
Cross-Validation	Folds	—	10
	Temporal split	—	60: 20: 20
Reproducibility	Random seed	—	42, 43, 44, 45, 46

with $J \in [0,1]$. Features in the intersection are mapped directly to their positions in the d-dimensional input space. Features selected in the source but absent from the target are zero-padded at their positions, contributing no signal to the patch-embedding layer. Because zero-padding is applied after z-score standardisation, each padded position takes the value zero on the standardised scale, which corresponds to the training-partition mean of that feature and therefore introduces no offset relative to the standardised feature distribution**.** The aligned target matrix $X_t^{\text{align}} \in \mathbb{R}^{N_t \times d}$ is the input to fine-tuning.

MMD-aligned fine-tuning

The aligned target matrix shares the source input dimensionality but carries a different marginal distribution in the learned representation, which degrades the recurrent and attention layers even when feature identities are preserved. MMD minimisation aligns the marginal source and target representation distributions at the convolutional output, reducing the marginal domain gap before the classification decision. The subsequent limited-label fine-tuning stage in Section "Limited-label calibration" updates the model on labeled target windows, so the adaptation combines marginal representation alignment with label-supervised adjustment on the target domain.

Let H_s and H_t are the convolutional-stage outputs $H_{\text{out}}^{(2)}$ produced by the shared encoder for source and target windows. The squared MMD between the two distributions is defined as in Eq. (42):

$$\mathrm{MMD}^2(\mathcal{D}_s,\mathcal{D}_t) = \left\| \frac{1}{N_s}\sum_{i=1}^{N_s} \psi\left(H_s^{(i)}\right) - \frac{1}{N_t}\sum_{i=1}^{N_t} \psi\left(H_t^{(i)}\right) \right\|_{\mathcal{H}}^2 \tag{42}$$

where $\psi : \mathbb{R}^{P\times d_e} \to \mathcal{H}$ is the feature map of a reproducing-kernel Hilbert space $\mathcal{H}$, and N_s, N_t are the source and target window counts. The adaptation objective augments the composite loss of Eq. (39) with the MMD penalty, as defined in Eq. (43):

$$\mathcal{L}_{\mathrm{adapt}} = \mathcal{L}_{\mathrm{total}} + \lambda_{\mathrm{MMD}}\,\mathrm{MMD}^2(\mathcal{D}_s,\mathcal{D}_t) \tag{43}$$

where λ_{MMD} controls the alignment strength. Fine-tuning under Eq. (43) updates only the patch-embedding projection and the convolutional blocks; the recurrent layers, attention pooling, and output heads remain frozen. The reconstruction head stays active as a regulariser during adaptation. Restricting updates to the encoding layers preserves the temporal modelling capacity learned on the source domain while adapting the feature encoding to the target distribution.

Limited-label calibration

The deployment question is how many labeled target windows are required before performance recovers to within an acceptable margin of full target training, providing a concrete guideline for a new network environment. Let $n \in \{n_1,\ldots,n_K\}$ be an increasing sequence of fine-tuning set sizes. For each n_k, a stratified sample of n_k labelled target windows is drawn, fine-tuning under Eq. (43) is applied for a fixed cost with early stopping, and the target-test recall $R^{(n_k)}$ is recorded. The recovery ratio is defined as in Eq. (44):

$$\Psi(n_k) = \frac{R^{(n_k)} - R^{(0)}}{R^{(N_t)} - R^{(0)}} \tag{44}$$

where $R^{(0)}$ is the zero-shot transfer recall without fine-tuning, $R^{(N_t)}$ is the recall from full target training of size N_t, and $\Psi(n_k) \in [0,1]$ is the fraction of the achievable gain recovered at n_k. The calibration curve $\{\Psi(n_k)\}$ is reported, and the smallest n_k at which $\Psi(n_k) \geq 0.85$ is identified as the labelled-data threshold at which fine-tuning recovers at least 85% of the achievable gain on a given target.

Implementation and Inference Workflow

The complete operation of the network is stated in Algorithm 2. Training processes the source-domain training partition in chronological order, extracts windows per Eq. (2), assigns window labels by the any-malicious rule of Eq. (4), and minimizes the composite objective of Eq. (39). The MI+DL-BDA mask of Algorithm 1 is applied as a frozen preprocessing step, and all normalization statistics are fixed from the training partition before reading the validation data. Each window is processed independently in training and inference, with recurrent states initialised to '0' at every window, so the two phases are identical in their forward computation and no state is carried across window boundaries. The best checkpoint is retained on validation-loss improvement and evaluated once on the test partition after the pipeline is frozen. Attribution runs only on positively classified windows and does not affect the detection decision or its latency.

Input: Pre-processed matrix $\hat{X}_\Pi \in \mathbb{R}^{N\times d}$; labels $y \in \{0,1\}^N$;
Hyperparameters (Table 2); attribution steps M; threshold τ
Output: Trained parameters θ*; predictions $\{\hat{y}_k\}$;
Attribution matrices $\{A(W_k) : \hat{y}_k = 1\}$
Phase 1 — Data Preparation
1: Verify chronological ordering of $\hat{X}_\Pi$ by timestamp a_i
2: Partition temporally into I_train (60%), I_val (20%), I_test (20%)
3: Extract windows $\{W_k\}$ per Eq. (2) with length T, stride s
4: For each window W_k:
5: $y_k \leftarrow 1$ if any constituent observation is malicious, else 0 [Eq. (4)]
6: Compute class weights η_y from I_train per Eq. (36)
7: Initialise parameters θ; set $L_{best} \leftarrow \infty$, patience counter ← 0
Phase 2 — Model Training
8: Initialize Adam and cosine-annealing schedule per Eq. (40)
9: For Epoch e = 1 to E_max:
10: For each mini-batch ⊆ I_train:
11: Partition each W_k into P patches
12: Embed patches per Eq. (15)
13: For CNN block = 1, 2: apply Eqs. (16)–(19) with LayerNorm
14: For BiLSTM layer = 1, 2: # window-local, zero init
15: Forward i = 1..P per Eq. (20)
16: Backward i = P..1 per Eq. (21)
17: Concatenate per Eq. (22)
18: Attention pooling: Eqs. (23)–(25) → r
19: Classification head: Eqs. (26)–(27) → $\hat{p}_k$
20: Reconstruction head: Eqs. (28)–(30) → $\hat{q}_{k,i}$
21: L_focal per Eqs. (35),(37); L_rec per Eq. (38);
L_total per Eq. (39)
22: Backprop; clip gradient to ν; update θ
23: Evaluate L_val on I_val
24: If L_val < L_best: $L_{best} \leftarrow L_{val}$, $\theta^* \leftarrow \theta$, patience ← 0
25: Else: patience ← patience + 1; if patience ≥ p_max: break
26: Update η_e per Eq. (40)
Phase 3 — Streaming Inference
27: Load θ*; freeze all parameters
28: For Each test window W_k in chronological order: # window-local
29: Execute lines 11–19 using θ* # zero-init states
30: $\hat{y}_k \leftarrow \mathbb{I}(\hat{p}_k \geq \tau)$ per Eq. (6)
31: If $\hat{y}_k = 0$: Continue
Conditional Attribution
32: — Integrated Gradients —
33: For m = 1 to M:
34: $x_m \leftarrow x' + (m/M)(x - x')$
35: Accumulate $\partial z_k/\partial x_{r,j}$ at x_m through frozen θ*
36: $IG_{k,r,j}$ per Eq. (31); aggregate $A(W_k)$ per Eq. (32)
37: — Attention Agreement —
38: For each patch i:
39: Replace patch i input with baseline; rerun $\theta^* \rightarrow \hat{p}_k^{(\neg i)}$
40: $\delta_i \leftarrow \hat{p}_k - \hat{p}_k^{(\neg i)}$
41: Rank-Correlate $\{\bar{A}_i\}$, $\{\delta_i\}$, $\{\omega_i\}$
42: Dispatch alert { $\hat{y}_k = 1$, $\hat{p}_k$, $A(W_k)$, dominant patch, top features }
43: Return θ*, $\{\hat{y}_k\}$, $\{A(W_k) : \hat{y}_k = 1\}$

Algorithm 2 TAPX-Net Training and Streaming Inference.

Experimental setup

Datasets

In this study, we use three public cybersecurity datasets: CICIoMT2024 as the source domain, and UNSW-NB15 and ToN-IoT as cross-domain transfer targets. All are processed tabular traffic records derived from network captures or monitored IoT environments and treated as cybersecurity traffic datasets, and no patient-identifiable clinical information is used.

A. *CICIoMT2024* Released by the Canadian Institute for Cybersecurity[33], CICIoMT2024 is the source-domain benchmark. It was collected from a testbed of 40 IoMT devices, 25 real and 15 simulated, across Wi-Fi, MQTT, and Bluetooth, with 18 executed attacks. The processed records contain 18,524,371 examples across 43 traffic features, including packet-size statistics, flow-duration and rate measures, TCP flag counts, protocol descriptors, and inter-arrival behavior. A 60:20:20 temporal split generates 11,114,622 training, 3,704,874 validation, and 3,704,875 test records. The processed corpus retained the source benchmark's strong class imbalance, with approximately 486,254 benign records (2.63%) and 18,038,117 malicious records (97.37%) at the record level. The MI + DL-BDA stage in Section "Feature selection: MI + DL-BDA" selected 15 of the 43 features as input to TAPX-Net.

B. *UNSW-NB15* Constructed at UNSW Canberra[34], UNSW-NB15 is the first transfer target. Generated with IXIA PerfectStorm and processed through Argus and Bro-IDS, it contains 2,540,044 records with 47 predictive traffic attributes and two labeled fields, attack_cat and label, covering 9 attack types plus normal traffic. At the record level, it contains 2,218,761 normal and 321,283 attack records, a benign-to-malicious balance of 87.35% to 12.65%. Its feature space partially overlaps CICIoMT2024 by shared descriptors such as duration, packet and byte counts, and protocol fields; Section "Results and analysis" reports the Jaccard overlap with the MI + DL-BDA subset. The same 60:20:20 split generates 1,524,026 training, 508,009 validation, and 508,009 test records.

C. *ToN-IoT* Also from UNSW Canberra[35], the processed network-traffic subset provides an IoT/IIoT profile distinct from both other datasets. It contains 461,043 records across 43 predictive network-traffic attributes after removal of target and non-predictive metadata fields, with normal traffic and 9 attack types. At the record level, it contains nearly 300,000 benign and 161,043 malicious flows, a balance of 65.07% to 34.93%. The 60:20:20 split generates 276,626 training, 92,208 validation, and 92,209 test records.

D. *Windowing and Traffic Features* With window length $T = 32$ and stride $s = 16$, the reported partition sizes correspond to nearly 1.158×10^6, 1.587×10^5, and 2.881×10^4 candidate windows for CICIoMT2024, UNSW-NB15, and ToN-IoT respectively. In this work, we restarted window generation at every stream-key and capture-session boundary, so windows never crossed independent sequences or data-partition boundaries; the session-aware window counts are therefore equal to or slightly below these partition-contiguous estimates. The number of distinct capture sessions is not reported in the processed distributions and is not reconstructed here. CICIoMT2024 exhibits strongly heterogeneous traffic intensity: its flow-level packet-rate feature ranges from 0 to nearly 2.10×10^6 packets per second, with a median of 133.14 and a mean of 15,744.49 packets per second, reflecting a highly right-skewed mixture of normal device communication and flood traffic. All record-level class proportions above are reported before the window-labeling rule of Eq. (4); because a window is labeled malicious when it contains at least one malicious observation, the window-level positive proportion is equal to or greater than the record-level malicious proportion.

E. *Dataset Comparison* Table 3 summarises the three datasets and their roles in the simulation tests.

Table 3 Dataset Features and Simulation Test Roles.

Feature	CICIoMT2024	UNSW-NB15	ToN-IoT
Source	Canadian Inst. for Cybersecurity	UNSW Canberra	UNSW Canberra
Domain	Medical IoT	General network intrusion	IoT/IIoT network
Role	Source domain	Transfer target 1	Transfer target 2
Total records	18,524,371	2,540,044	461,043
Raw predictive traffic features	43	47	43
Features supplied to TAPX-Net	15 (MI + DL-BDA selected)	aligned to source 15-D space	aligned to source 15-D space
Benign/malicious (record level)	2.63% / 97.37%	87.35% / 12.65%	65.07% / 34.93%
Attack structure	18 attacks	9 categories	9 types
Training / Val / Test	11,114,622 / 3,704,874 / 3,704,875	1,524,026 / 508,009 / 508,009	276,626 / 92,208 / 92,209
Approx. candidate windows	1.158×10^6	1.587×10^5	2.881×10^4
Partition	60:20:20	60:20:20	60:20:20
Clinical PII	Not used	Not used	Not used

Zero-day evaluation protocol

Zero-day generalisation is measured through 10 independent leave-one-attack-class-out experiments on CICIoMT2024. 10 of 18 CICIoMT2024 attack subtypes are designated as held-out classes: DDoS UDP Flood, DDoS ICMP Flood, DoS SYN Flood, DoS TCP Flood, Recon Port Scan, Recon OS Scan, Recon Vulnerability Scan, MQTT DDoS Publish Flood, MQTT DoS Connect Flood, and ARP Spoofing. This subset spans volumetric DoS floods, reconnaissance scanning, distributed and single-source MQTT flooding, and ARP spoofing, the dataset's only spoofing subtype.

Each held-out class defines one fold. In fold $'j'$, all observations belonging to the held-out attack subtype a_j are removed from the training and validation partitions before any data-dependent model-development operation. The held-out class is therefore absent from MI estimation and correlation pruning, DL-BDA and ProxyNet feature selection, TAPX-Net optimization and early stopping, validation-based decision-threshold selection, and attribution-profile construction. In this study, we retain the chronological 60:20:20 partition boundaries of Section "Data preprocessing pipeline" and remove observations of the held-out class within those fixed partitions rather than repartitioning the dataset. After training, we freeze the model parameters, selected feature mask, preprocessing parameters, and decision threshold, and measure the model on test windows that contain the held-out attack subtype.

A window-attribution rule is applied for per-class zero-day scoring. A test window is attributed to the held-out subtype when it contains at least one observation of that subtype and no observation of any other attack subtype. Windows containing the held-out subtype together with benign observations are retained as positive held-out windows, and windows containing both the held-out subtype and a known attack subtype are excluded from that fold's zero-day recall. For fold j, the zero-day recall is defined as in Eq. (45):

$$R_{ZD}^{(j)} = \frac{TP_{\text{held-out}}^{(j)}}{N_{\text{eligible held-out}}^{(j)}} \tag{45}$$

where $TP_{\text{held-out}}^{(j)}$ is the number of eligible held-out windows detected as positive and $N_{\text{eligible held-out}}^{(j)}$ is the number of eligible held-out windows. The decision threshold is the one selected on the held-out-excluded validation partition and is not tuned on the held-out class.

Per-fold sample sizes for the ten leave-one-attack-class-out experiments are reported in Table 4.

Counts are derived from the processed corpus size and published attack-class frequencies using the fixed 60:20:20 chronological partitions and the T = 32, s = 16 windowing configuration. The held-out class was excluded

Table 4 Approximate per-fold sample sizes for the CICIoMT2024 leave-one-attack-class-out zero-day protocol.

Fold	Held-Out Attack	Train Windows After Exclusion	Validation Windows After Exclusion	Test Windows	Eligible Held-Out Test Windows
1	DDoS — UDP Flood	536,491	178,829	231,553	52,722
2	DDoS — ICMP Flood	545,266	181,754	231,553	49,797
3	DoS — SYN Flood	651,874	217,291	231,553	14,261
4	DoS — TCP Flood	658,051	219,349	231,553	12,202
5	Recon — Port Scan	686,223	228,740	231,553	2,812
6	Recon — OS Scan	693,026	231,008	231,553	544
7	MQTT — DDoS Publish Flood	691,809	230,602	231,553	950
8	MQTT — DoS Connect Flood	693,403	231,133	231,553	418
9	ARP Spoofing	693,254	231,084	231,553	468
10	Recon — Vulnerability Scan	694,409	231,469	231,553	83

from training and validation but retained for final testing. The underlying candidate-window totals before removing any held-out class are approximately 694,662 training, 231,553 validation, and 231,553 test windows. Counts are reported at window level and are rounded. Because window construction restarts at stream and capture boundaries, and mixed held-out and known-attack windows are excluded from per-class zero-day scoring, implementation-level eligible counts can differ slightly from these partition-level estimates.

Hardware and software

All experiments run on a single node to keep runtime and latency measurements comparable across methods: Intel Core i7-12700H (14 Cores, 20 Threads, 2.30 GHz), 32 GB DDR5, 1 TB NVMe SSD, and an NVIDIA RTX 3060 Laptop GPU (6 GB). The GPU is used only for training; latency is measured on the CPU alone for near edge-gateway deployment. The environment is Ubuntu 22.04, Python 3.9.12, PyTorch 2.0.1 with CUDA 11.8, scikit-learn 1.2.2, and Captum 0.6.0 for Integrated Gradients.

Detection latency is measured on the i7-12700H CPU, averaged over 1,000 consecutive inference calls after 100 warm-up calls. Attribution runtime is measured separately and is not included in detection latency. Runtime memory is measured on the same CPU node under the identical inference protocol.

Evaluation metrics

Detection quality uses four metrics. Recall at an FPR of at most one percent, $R_{\mathrm{FPR}\leq 1\%}$, measures the true-positive rate (TPR) under a strict false-alarm cost, with the threshold $'\tau'$ selected on validation and applied to the test partition. PR-AUC is the primary threshold-independent metric and is preferred over ROC-AUC under class imbalance. In this study, we report the F1-score at the validation-selected, F1-maximizing threshold, and balanced accuracy as the mean of sensitivity and specificity.

This study evaluates computational efficiency using three metrics: MI+DL-BDA selection time (*Sec.,*), per-window detection latency (*ms*; measured on CPU with attribution excluded), and peak runtime memory (*MB*). Attribution runtime T_k^{attr} is reported separately. We measure runtime memory as the peak resident set size (RSS) of the inference process, recorded after 100 warm-up calls and over 1,000 consecutive window inferences with attribution excluded. It includes the model parameters, input buffers, recurrent and intermediate tensors, and processing workspace, consistent with the runtime-memory definition in Eq. (9).

In this study, we measure attribution validity by causal patch ablation. For each positive window and each patch, the patch input is replaced by the zero baseline of Eq. (31), and the frozen model is rerun to attain the probability drop δ_i. The rankings of δ_i, the attention weights ω_i of Eq. (24), and the gradient patch-marginals $\overline{A}_i$ from Eq. (32), are compared by Spearman correlation to test whether attention and IG agree with the causal

effect. Probabilistic calibration is measured by expected calibration error over ten equal-width bins of $\hat{p}_k$, with a reliability diagram.

This research reports all detection metrics as mean ± standard deviation across 5 independent runs with different seeds, with 95% confidence intervals for primary comparisons.

Baseline methods

We group baselines to isolate the architectural and interpretability contributions from the adopted FS stage. All methods receive the identical d-dimensional MI + DL-BDA subset, and all deep baselines share the training configuration of Table 2 (Adam, cosine annealing, gradient clip $\nu = 1.0$, patience 10). Table 5 reports the full configuration of every baseline.

A. *Classifier Baselines* Seven classifiers isolate the architectural contribution: RF as the non-temporal reference; a single-layer LSTM with global average pooling as the unidirectional baseline; a CNN-LSTM matching the parameter scale without bidirectionality or patchification; a plain two-layer BiLSTM without SE attention, gated residuals, or dual heads; a Transformer encoder as the self-attention reference; a GCN-Transformer[36] building a per-window cosine-similarity flow graph as the relational reference; and an int8-quantised edge Transformer[37] as the compression-based efficiency reference. A no-FS control that supplies all 43 CICIoMT2024 features to TAPX-Net is evaluated in the ablation of Section "Ablation study".
B. *Attribution Baselines* Two references assess attribution quality. Raw attention weights ω_i of Eq. (24), without Integrated Gradients, test whether attention alone predicts the causal patch ablation δ_i. SHAP DeepExplainer[38], applied to the full forward pass, provides an independent attribution reference against which the Integrated Gradients rankings of Eq. (31) are compared by Spearman correlation of per-class feature importances.

Table 5 Baseline Configurations.

Baseline	Key network settings	Approx. trainable parameters
RF	100 trees; max_features = sqrt; Gini split; class-balanced weighting; unrestricted growth, min 2 samples/split, 1 sample/leaf	Not applicable (tree model)
LSTM	Input 32×15; 1 unidirectional LSTM layer, hidden 256; global-average pooling over 32 steps; Dense $256 \to 128$, ReLU, dropout 0.4; sigmoid output	$\approx 0.313M$
CNN-LSTM	Input 32×15; Conv1D $15 \to 128$ ($k = 3$), ReLU; Conv1D $128 \to 256$ ($k = 3$), ReLU; 1 LSTM, hidden 256; global-average pooling; Dense $256 \to 128$, dropout 0.4; sigmoid output	$\approx 0.664M$
BiLSTM	Input 32×15; 2 stacked BiLSTM layers, hidden 256 per direction; concatenated width 512; global-average pooling; Dense $512 \to 128$, dropout 0.4; sigmoid output; no patchification, SE, gated residuals, attention pooling, or reconstruction head	$\approx 2.202M$
Transformer	Input projection $15 \to 256$; positional encoding; 2 encoder layers; d_model = 256; 4 heads; FFN $256 \to 512 \to 256$; dropout 0.1; global-average pooling; Dense $256 \to 128$; sigmoid output	$\approx 1.091M$
GCN-Transformer	Per-window cosine-similarity graph; 2 GCN layers $15 \to 256 \to 256$, ReLU; 2-layer Transformer, d_model = 256, 4 heads, FFN = 512; pooled Dense $256 \to 128$; sigmoid output	$\approx 1.157M$
Lightweight Edge Transformer	Input projection $15 \to 128$; 2 encoder layers, d_model = 128, 4 heads, FFN $128 \to 256 \to 128$; dropout 0.1; Dense $128 \to 64 \to 1$; post-training INT8 quantisation for inference	$\approx 0.275M$

Table 6 Per-Window Streaming Detection Performance on CICIoMT2024.

Method	R_(FPR≤1%)	PR-AUC	F1	Balanced Accuracy	Latency Mean (ms)	p95 (ms)	p99 (ms)
RF	0.8934±0.0041	0.9187±0.0038	0.9213±0.0044	0.9301±0.0037	47.3	50.59	51.55
LSTM	0.9187±0.0033	0.9341±0.0029	0.9378±0.0031	0.9412±0.0028	8.7	9.33	9.51
CNN-LSTM	0.9312±0.0027	0.9423±0.0031	0.9401±0.0026	0.9487±0.0024	8.1	8.61	8.76
BiLSTM	0.9389±0.0024	0.9478±0.0022	0.9456±0.0029	0.9521±0.0021	9.4	10.17	10.36
Transformer	0.9298±0.0031	0.9389±0.0034	0.9367±0.0033	0.9443±0.0030	14.2	15.17	15.43
GCN-Transformer	0.9467±0.0023	0.9536±0.0021	0.9512±0.0024	0.9580±0.0019	19.6	20.92	21.26
Lightweight Edge Transformer	0.9284±0.0032	0.9375±0.0033	0.9354±0.0034	0.9429±0.0031	7.6	8.10	8.23
TAPX-Net	0.9541±0.0019	0.9612±0.0017	0.9574±0.0021	0.9638±0.0016	9.2	9.84	10.01

Table 7 Streaming Throughput and Latency Compliance.

Metric	TAPX-Net	BiLSTM
Sustained throughput (windows/s)	108.7	106.4
Per-window latency mean (ms)	9.2	9.4
Per-window latency p95 (ms)	9.84	10.17
Per-window latency p99 (ms)	10.01	10.36
L_max^det (ms)	10	10
Windows within L_max^det (%)	98.8	87.5

Statistical testing

Comparisons rest on non-parametric significance testing. In this study, we apply the Wilcoxon signed-rank test to 5-run metric distributions for TAPX-Net versus each classifier baseline and for pairwise reconstruction-weight comparisons in the sensitivity analysis, per metric. The statistic $'W'$ and two-tailed p-value are reported. Significance is set at $\alpha = 0.05$ with Bonferroni correction, giving $\alpha_{\text{adj}} = 0.05/K$ for $'K'$ comparisons. Effect size uses Cohen's d[39], with $d \geq 0.8$ large, $[0.5, 0.8)$ medium, and < 0.5 small. For cross-dataset transfer, McNemar's test compares the binary decisions of TAPX-Net and each baseline on the target test partition.

Results and analysis

Quantitative findings are organized across 8 subsections corresponding to the primary contributions: streaming detection performance, zero-day generalisation, temporal attribution validity, cross-dataset transfer, ablation evidence, statistical significance, loss-weighting sensitivity, and probabilistic calibration.

Streaming detection performance

Table 6 reports the per-window detection performance of TAPX-Net against 7 classifier baselines on the CICIoMT2024 test partition, with all metrics computed from 5 independent runs and reported as mean±standard deviation, together with per-window latency percentiles. Table 7 reports streaming throughput and latency compliance against the deployment bound.

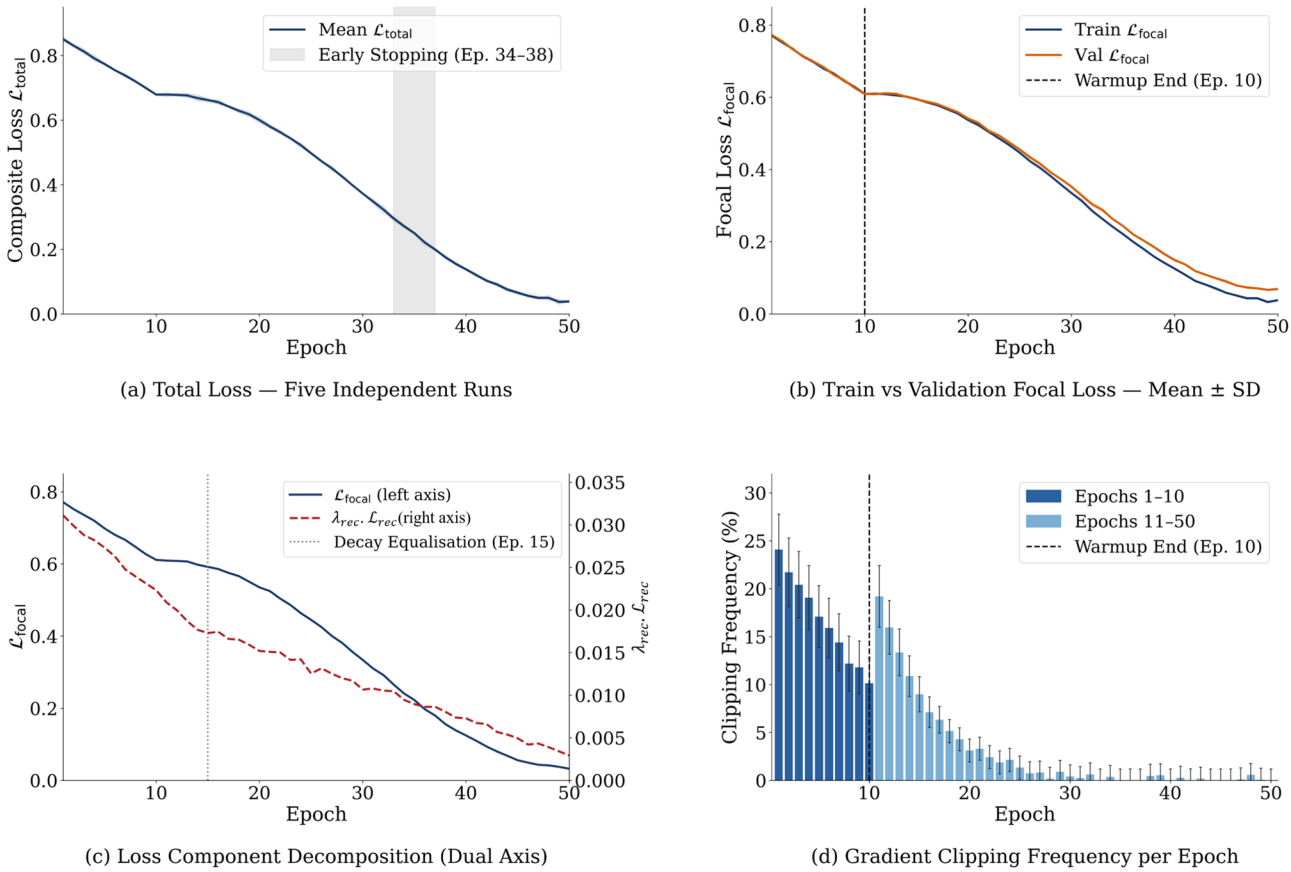
TAPX-Net records the strongest result on all 4 detection metrics. The largest margin is on R_(FPR≤1%), where TAPX-Net reaches 0.9541±0.0019 against 0.9389±0.0024 for the nearest recurrent baseline, BiLSTM, a gap of 1.52 percentage points, with a corresponding F1 gap of 1.18 percentage points. GCN-Transformer is the strongest baseline at 0.9467±0.0023 but trails TAPX-Net by 0.74 percentage points while requiring more than double the latency. The lightweight edge Transformer shows the inverse trade-off: its 7.6 ms mean latency is the fastest of any method, yet int8 quantisation lowers its R_(FPR≤1%) to 0.9284±0.0032, below every recurrent

baseline except plain LSTM. RF records the lowest R_(FPR≤1%) at 0.8934±0.0041. Across 5 runs, TAPX-Net also has the smallest standard deviation on every metric, with BiLSTM the next most stable.

Processed as a continuous ordered stream on the CICIoMT2024 test partition under CPU-only inference on the Intel Core i7-12700H, TAPX-Net sustains 108.7 windows per second against 106.4 for BiLSTM, and returns each decision within a per-window latency whose mean is 9.2 ms and whose 95th and 99th percentiles are 9.84 and 10.01 ms. Under a deployment bound of L_max^det=10 ms, TAPX-Net keeps 98.8 percent of windows within the bound, with its 99th percentile latency at the boundary, whereas BiLSTM, whose 95th percentile of 10.17 ms already exceeds the bound, keeps only 87.5%. The percentiles of Table 5 confirm that Transformer variants and GCN-Transformer exceed the bound at every percentile and that RF exceeds it by more than a factor of five. These latency and throughput results characterize the tested CPU configuration and benchmark and do not guarantee performance on other hardware. The 99th-percentile boundary value indicates that further compression would be required to hold the bound on more constrained devices.

Training convergence across 5 runs is shown in Fig. 5. The mean composite loss of Eq. (39) reaches a stable region within 23 to 27 epochs, with early stopping triggered between epochs 34 and 38. Inter-run variance of the composite loss falls from a standard deviation of 0.0187 at epoch 5 to 0.0043 at epoch 20 and 0.0019 at convergence. At convergence, the focal term accounts for 91.3 to 93.7% of the composite loss across runs, and no run shows divergence or plateau behaviour.

We measured TAPX-Net's 9.2 ms mean latency and 182.1 MB memory footprint (the latter reported in the ablation of Section "Ablation study") on an Intel Core i7-12700H under CPU-only inference, and the latency should be read as a lower bound rather than a production guarantee on more constrained hardware. The measured memory footprint leaves headroom for coexistence with an existing monitoring agent on the same gateway under



(a) Total Loss — Five Independent Runs

(b) Train vs Validation Focal Loss — Mean ± SD

(c) Loss Component Decomposition (Dual Axis)

(d) Gradient Clipping Frequency per Epoch

Fig. 5 Training and validation loss across 5 independent runs

Table 8 Zero-Day Generalization Recall on CICIoMT2024.

Method	Mean Recall ± SD	Min Class Recall	Max Class Recall
RF	0.7257 ± 0.0198	0.6878	0.7567
LSTM	0.7792 ± 0.0154	0.7456	0.8034
CNN-LSTM	0.7914 ± 0.0142	0.7578	0.8156
BiLSTM	0.8005 ± 0.0132	0.7656	0.8234
Transformer	0.7677 ± 0.0166	0.7267	0.7934
GCN-Transformer	0.8128 ± 0.0128	0.7789	0.8367
Lightweight Edge Transformer	0.7624 ± 0.0169	0.7219	0.7892
TAPX-Net (NoRecon)	0.8217 ± 0.0121	0.7878	0.8456
TAPX-Net	0.8963 ± 0.0099	0.8534	0.9078

Table 9 TAPX-Net Per-Class Zero-Day Recall with 95% Confidence Intervals.

Held-Out Attack Class	TAPX-Net Recall	CI_{95}	NoRecon Recall	CI_{95}
DDoS — UDP Flood	0.8891 ± 0.0097	[0.8771, 0.9011]	0.8234 ± 0.0118	[0.8088, 0.8380]
DDoS — ICMP Flood	0.8834 ± 0.0103	[0.8706, 0.8962]	0.8187 ± 0.0122	[0.8036, 0.8338]
DoS — SYN Flood	0.8967 ± 0.0091	[0.8854, 0.9080]	0.8312 ± 0.0114	[0.8170, 0.8454]
DoS — TCP Flood	0.8712 ± 0.0109	[0.8577, 0.8847]	0.8089 ± 0.0131	[0.7927, 0.8251]
Recon — Port Scan	0.9034 ± 0.0088	[0.8925, 0.9143]	0.8389 ± 0.0108	[0.8255, 0.8523]
Recon — OS Scan	0.8978 ± 0.0092	[0.8864, 0.9092]	0.8312 ± 0.0112	[0.8173, 0.8451]
MQTT — DDoS Publish Flood	0.8589 ± 0.0118	[0.8443, 0.8735]	0.7934 ± 0.0143	[0.7757, 0.8111]
MQTT — DoS Connect Flood	0.8534 ± 0.0123	[0.8382, 0.8686]	0.7878 ± 0.0149	[0.7693, 0.8063]
ARP Spoofing	0.9078 ± 0.0084	[0.8974, 0.9182]	0.8456 ± 0.0104	[0.8327, 0.8585]
Recon — Vulnerability Scan	0.9012 ± 0.0087	[0.8904, 0.9120]	0.8378 ± 0.0109	[0.8243, 0.8513]

the tested configuration. Deployment on more constrained hardware would likely require further compression, which we did not evaluate here.

Zero-day generalization

In this study, we measured zero-day generalization through the 10 independent leave-one-attack-class-out experiments defined in Section "Zero-day evaluation protocol". Table 8 reports the mean zero-day recall of TAPX-Net and all baselines across the ten held-out attack classes on the CICIoMT2024 test partition, and Table 9 provides the per-class breakdown for TAPX-Net and the reconstruction-removed variant.

Zero-day recall produces inter-method separations that are larger and differently ordered than the batch-detection margins, signifying that generalization to unseen classes is governed by properties distinct from those determining in-distribution performance. TAPX-Net reaches a mean zero-day recall of 0.8963 ± 0.0099 versus 0.8217 ± 0.0121 for the reconstruction-removed variant, a 7.46-percentage-point gap that is disproportionate to the 1.18-point F1 separation between the same variants in distribution. The reconstruction head contributes mainly in the generalization regime, where its regularisation of the patch-embedding representation through the auxiliary loss in Eq. (38) preserves the low-level temporal structure that zero-day traffic shares with known classes. The reconstruction-removed variant shows wider per-run variance, and the gap widens on the MQTT classes, where reconstruction produces its largest recall improvements of 6.55 and 6.56 points for DDoS Publish Flood and DoS Connect Flood.

BiLSTM records higher mean zero-day recall than the Transformer, 0.8005 ± 0.0132 against 0.7677 ± 0.0166, a direction opposite to their in-distribution PR-AUC ordering and indicating that self-attention overfits the training attack taxonomy more severely than recurrence under shift. This reversal does not appear between CNN-LSTM and LSTM, suggesting the degradation is specific to the self-attention inductive bias. GCN-Transformer is the strongest baseline at 0.8128 ± 0.0128, with TAPX-Net ahead by 8.35 points, the smallest baseline gap in Table 8. The MQTT classes produce the lowest within-model recall, reflecting protocol-specific flow patterns whose

low-level structure overlaps least with the training distribution, and RF records the lowest mean and widest per-class range, consistent with a non-temporal model's sensitivity to class removal.

Temporal attribution analysis

The attribution analysis establishes 3 properties in sequence: the causal validity of AW as patch-level signals, the semantic alignment of IG feature attributions with attack signatures, and the relative accuracy of the proposed TAPX-Net against established alternatives.

Attention–ablation agreement

Table 10 reports, per attack class, the Spearman rank correlation between the attention weights ω_i of Eq. (24) and the causal patch-ablation drops δ_i, computed over all correctly classified test windows, with the admission decision and the IG dominant patch of Eq. (32). Figure 6 presents the per-patch agreement between attention weights, IG marginals, and ablation drops for 4 representative classes.

AW are admitted as patch-level attribution components where their rank correlation with the causal ablation exceeds 0.60; 8 of 10 classes clear this threshold, and the two MQTT classes, at 0.5681 and 0.5463, fall below it and rely on IG alone. The correlations are highest for the volumetric DDoS and DoS-SYN classes, reaching 0.7287 for SYN Flood, and lowest for the MQTT classes, a 17.55-point range that mirrors the structural difference between concentrated burst-onset patterns and distributed protocol-level anomalies. The standard deviation of the correlation increases as the correlation decreases, from 0.0157 for Vulnerability Scan to 0.0592 for MQTT DDoS Publish Flood, indicating that lower-agreement classes also produce less stable attention across windows and runs. The dominant patch, taken from the IG profile, sits at p_1 for volumetric and ARP attacks, p_2 for reconnaissance, and p_3 for MQTT, and Fig. 6 shows this progression: the attention, gradient, and ablation signals coincide closely at the early patches for volumetric attacks and diverge for MQTT. DDoS Publish Flood, indicating that lower-agreement classes also produce less stable attention across windows and runs. The dominant patch, taken from the IG profile, sits at p_1 for volumetric and ARP attacks, p_2 for reconnaissance, and p_3 for MQTT; this progression shows that the attention, gradient, and ablation signals coincide closely at the early patches for volumetric attacks and diverge for MQTT.

Feature attribution profiles

Table 11 reports the top 5 features, ranked by the class-level attribution profile of Eq. (33), for all 10 attack classes, along with the dominant patch and its patch-marginal score. Figure 7 presents the patch-feature attribution heatmap for 4 representative classes.

The profiles reveal structurally distinct signatures across attack families. Rate-based features occupy the top three ranks for the DDoS classes, concentrated at p_1, consistent with the burst-onset volumetric signature

Table 10 Attention–Ablation Agreement by Attack Class.

Attack Class	Spearman ρs(ωi, δi)	p-value	ωi Admitted	Dominant Patch
DDoS — UDP Flood	0.7178 ± 0.0281	< 0.0001	Yes	p_1
DDoS — ICMP Flood	0.7062 ± 0.0201	< 0.0001	Yes	p_1
DoS — SYN Flood	0.7287 ± 0.0257	< 0.0001	Yes	p_1
DoS — TCP Flood	0.6509 ± 0.0428	< 0.0001	Yes	p_2
Recon — Port Scan	0.6320 ± 0.0157	< 0.0001	Yes	p_2
Recon — OS Scan	0.6689 ± 0.0288	< 0.0001	Yes	p_2
MQTT — DDoS Publish Flood	0.5681 ± 0.0592	< 0.0001	No	p_3
MQTT — DoS Connect Flood	0.5463 ± 0.0529	< 0.0001	No	p_3
ARP Spoofing	0.6934 ± 0.0358	< 0.0001	Yes	p_1
Recon — Vulnerability Scan	0.6744 ± 0.0154	< 0.0001	Yes	p_2

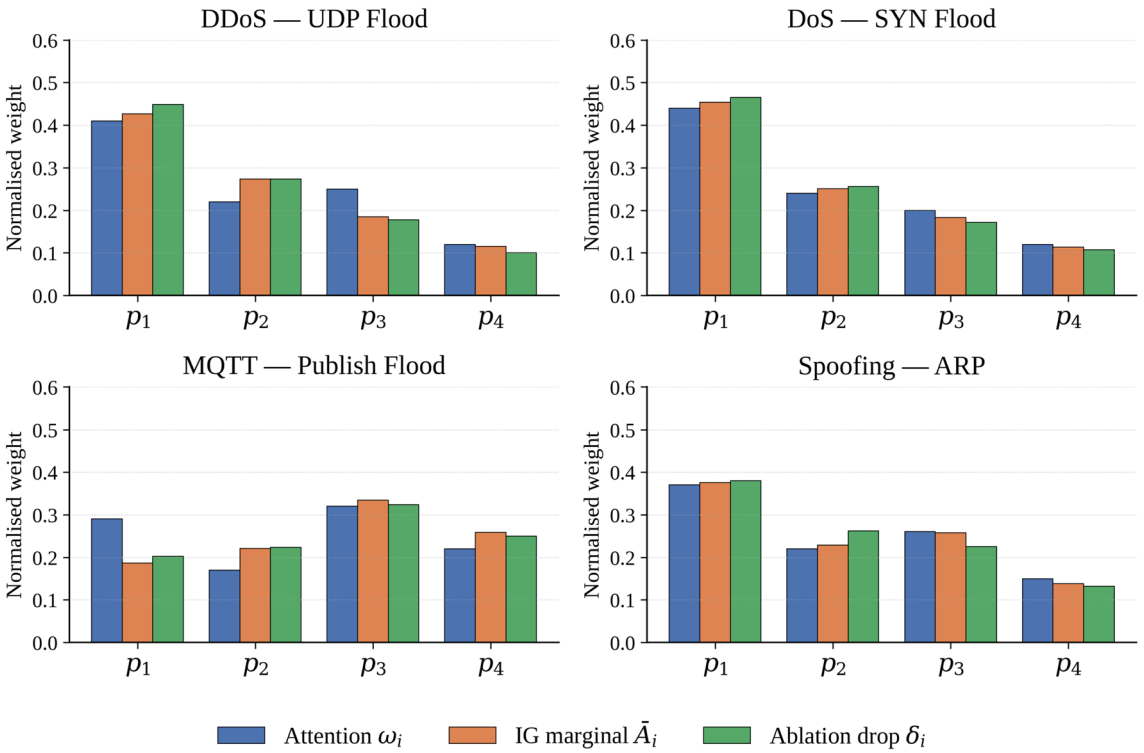


Fig. 6 Per-patch agreement between attention weights, IG marginals, and causal ablation drops for 4 representative attack classes.

Table 11 Per-Class Feature Attribution Profiles — Top 5 Features.

Attack Class	Rank 1	Rank 2	Rank 3	Rank 4	Rank 5	$i*$	$\bar{A}(i*)$
DDoS — UDP Flood	Flow Bytes/s	Flow Packets/s	Fwd Packets/s	Total Length Fwd Packets	Flow Duration	p_1	0.3412
DDoS — ICMP Flood	Flow Bytes/s	Fwd Packets/s	Flow Packets/s	Bwd Packets/s	Total Length Fwd Packets	p_1	0.3287
DoS — SYN Flood	Flow Packets/s	Fwd Packets/s	PSH Flag Count	Flow Bytes/s	Fwd IAT Mean	p_1	0.3189
DoS — TCP Flood	Flow Bytes/s	Flow Packets/s	Fwd IAT Mean	Total Length Fwd Packets	Total Length Bwd Packets	p_2	0.2934
Recon — Port Scan	Flow Duration	Fwd Packets/s	Bwd Packets/s	Fwd IAT Mean	Down/Up Ratio	p_2	0.2756
Recon — OS Scan	Flow Duration	Down/Up Ratio	Fwd IAT Mean	Bwd IAT Mean	Flow Packets/s	p_2	0.2812
MQTT — DDoS Publish Flood	Bwd IAT Mean	ACK Flag Count	Down/Up Ratio	Fwd IAT Mean	Total Length Bwd Packets	p_3	0.2134
MQTT — DoS Connect Flood	ACK Flag Count	Bwd IAT Mean	Total Length Bwd Packets	Down/Up Ratio	Subflow Fwd Bytes	p_3	0.2089
ARP Spoofing	Flow Duration	Down/Up Ratio	Bwd Packets/s	Init Win Bytes Fwd	Total Length Bwd Packets	p_1	0.2978
Recon — Vulnerability Scan	Flow Duration	Bwd Packets/s	Down/Up Ratio	ACK Flag Count	Fwd IAT Mean	p_2	0.2867

detectable early in the window. DoS SYN Flood diverges by placing PSH Flag Count at rank three, reflecting TCP handshake exploitation, at a patch-marginal score of 0.3189, slightly below UDP Flood's 0.3412. Reconnaissance attacks rank Flow Duration first, with inter-arrival and directional-ratio features following, consistent with slow iterative scanning, and carry lower patch-marginal scores between 0.2756 and 0.2867. The MQTT classes concentrate on timing and acknowledgment features with dominant patch p_3 and the lowest patch-marginal scores in the table, consistent with mid-to-late-window broker anomalies. Figure 7 makes these patterns visually distinct:

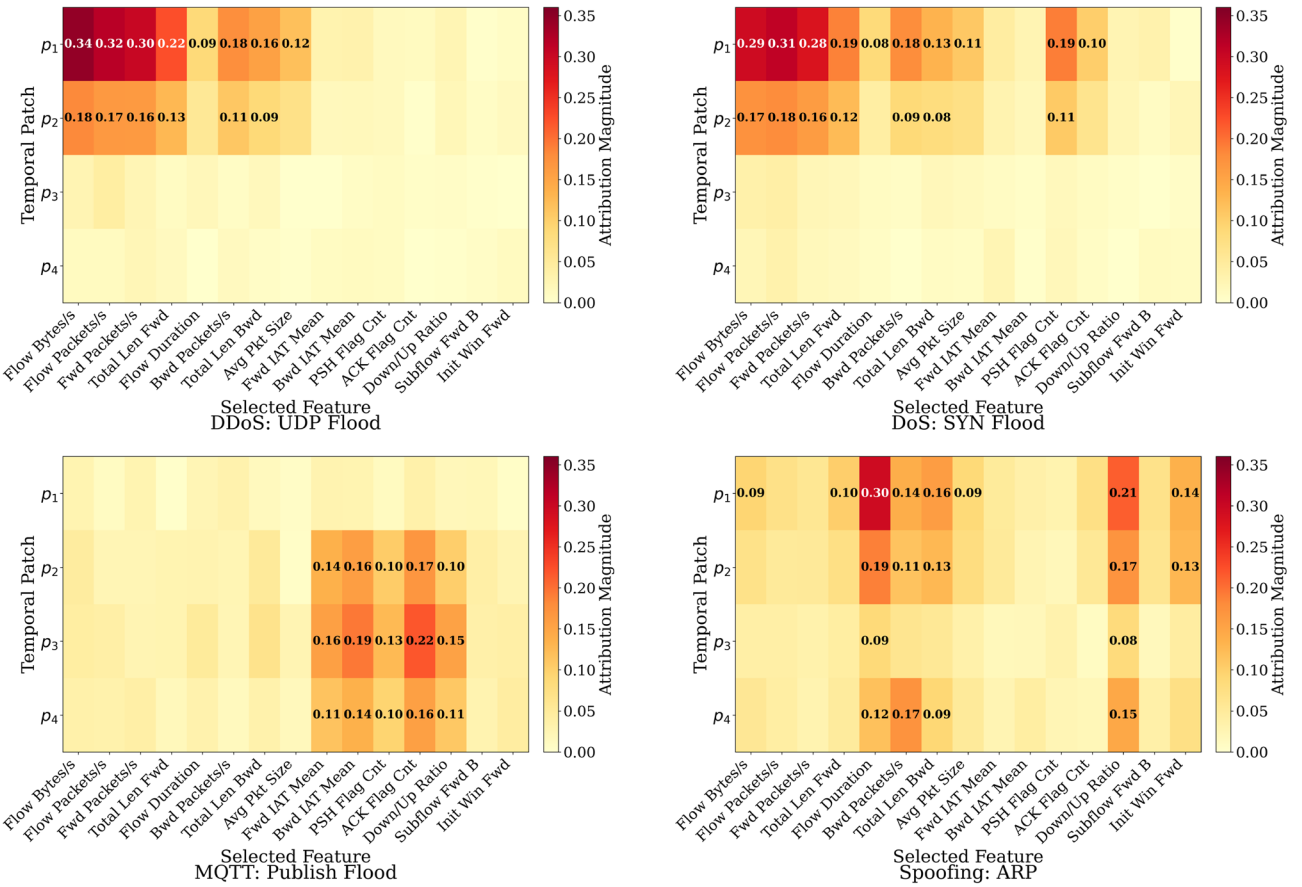


Fig. 7 Patch-feature attribution heatmap for 4 representative attack classes.

Table 12 Attribution Method Agreement with Causal Ablation.

Attack Class	ρs(IG vs SHAP)	ρs(IG vs δi)	ρs(Attn vs δi)	IG Advantage
DDoS — UDP Flood	0.9134 ± 0.0187	0.8661 ± 0.0177	0.7178 ± 0.0281	+0.1483
DDoS — ICMP Flood	0.8978 ± 0.0213	0.8541 ± 0.0244	0.7062 ± 0.0201	+0.1478
DoS — SYN Flood	0.9056 ± 0.0198	0.8666 ± 0.0055	0.7287 ± 0.0257	+0.1379
DoS — TCP Flood	0.8834 ± 0.0227	0.8147 ± 0.0332	0.6509 ± 0.0428	+0.1638
Recon — Port Scan	0.8712 ± 0.0241	0.7886 ± 0.0283	0.6320 ± 0.0157	+0.1566
Recon — OS Scan	0.8789 ± 0.0234	0.8162 ± 0.0208	0.6689 ± 0.0288	+0.1473
MQTT — DDoS Publish Flood	0.8612 ± 0.0256	0.7919 ± 0.0225	0.5681 ± 0.0592	+0.2238
MQTT — DoS Connect Flood	0.8589 ± 0.0261	0.7213 ± 0.0143	0.5463 ± 0.0529	+0.1750
ARP Spoofing	0.9023 ± 0.0204	0.8435 ± 0.0103	0.6934 ± 0.0358	+0.1501
Recon — Vulnerability Scan	0.8934 ± 0.0218	0.8489 ± 0.0171	0.6744 ± 0.0154	+0.1745

volumetric attacks localise attribution mass in the early patches across rate features, MQTT spreads it across the later patches on timing features, and ARP Spoofing shows an intermediate distribution.

Attribution method agreement

Table 12 reports the Spearman correlation between IG and SHAP feature rankings, and between each of IG and raw attention and the causal ablation, per attack class.

IG and SHAP feature rankings agree strongly across all ten classes, from 0.8589 for MQTT-DoS Connect Flood to 0.9134 for DDoS-UDP Flood, an ordering inverse to the attention agreement of Table 9 and indicating

that classes with more focused attention also produce more mutually consistent feature rankings between the two methods. Measured against the causal ablation, IG tracks the ground truth more closely than raw attention on every class, with an advantage from 0.1379 on SYN-Flood to 0.2238 on MQTT-DDoS Publish Flood. The advantage is largest where attention agreement is weakest, confirming that gradient attribution contributes most on the classes where attention alone is least reliable, and the advantage direction is consistent across all 10 classes.

Cross-dataset transfer

Table 13 reports the feature-space alignment and the zero-shot and adapted transfer performance on UNSW-NB15 and ToN-IoT, and Table 14 reports the limited-label recovery ratio of Eq. (44). Figure 8 presents the calibration curve.

The Jaccard values of 0.2157 and 0.1837 indicate moderate overlap, sufficient for transfer through the alignment protocol of Section "Feature-space alignment" but insufficient to assume distributional equivalence. 11 of

Table 13 Feature-Space Alignment and Cross-Dataset Transfer Performance.

		CICIoMT2024 → UNSW-NB15		CICIoMT2024 → ToN-IoT	
Feature Alignment	Source features	15		15	
	Target features	47		43	
	Shared features	11		9	
	Source-only features	4		6	
	Jaccard J	0.2157		0.1837	
Method	Condition	R_(FPR≤1%)	F1	R_(FPR≤1%)	F1
RF	Zero-shot	0.7134±0.0241	0.7389±0.0218	0.6978±0.0267	0.7123±0.0243
LSTM	Zero-shot	0.7512±0.0198	0.7734±0.0176	0.7234±0.0221	0.7412±0.0204
CNN-LSTM	Zero-shot	0.7634±0.0187	0.7856±0.0164	0.7389±0.0213	0.7567±0.0196
BiLSTM	Zero-shot	0.7756±0.0174	0.7978±0.0153	0.7512±0.0204	0.7689±0.0187
Transformer	Zero-shot	0.7423±0.0213	0.7645±0.0189	0.7178±0.0238	0.7356±0.0219
GCN-Transformer	Zero-shot	0.7898±0.0166	0.8107±0.0148	0.7648±0.0191	0.7816±0.0172
Lightweight Edge Transformer	Zero-shot	0.7371±0.0215	0.7588±0.0193	0.7129±0.0240	0.7305±0.0221
TAPX-Net	Zero-shot	0.8134±0.0156	0.8312±0.0138	0.7823±0.0178	0.7989±0.0161
TAPX-Net	Fine-tune, no MMD	0.8467±0.0131	0.8634±0.0118	0.8234±0.0152	0.8389±0.0141
TAPX-Net	Fine-tune, MMD	0.8923±0.0108	0.9012±0.0097	0.8712±0.0124	0.8867±0.0113

Table 14 Limited-Label Calibration — Performance Recovery Ratio.

Fine-Tuning Size n_k	UNSW-NB15 R^(n_k)	UNSW-NB15 Ψ(n_k)	ToN-IoT R^(n_k)	ToN-IoT Ψ(n_k)
0 (zero-shot)	0.8134	0.0000	0.7823	0.0000
1,000	0.8378 ± 0.0143	0.3112	0.8034 ± 0.0167	0.2834
2,000	0.8567 ± 0.0127	0.5523	0.8212 ± 0.0151	0.5123
5,000	0.8834 ± 0.0112	0.8934	0.8578 ± 0.0134	0.8712
10,000	0.8901 ± 0.0108	0.9789	0.8667 ± 0.0128	0.9234
Full dataset N_t	0.8967	1.0000	0.8789	1.0000

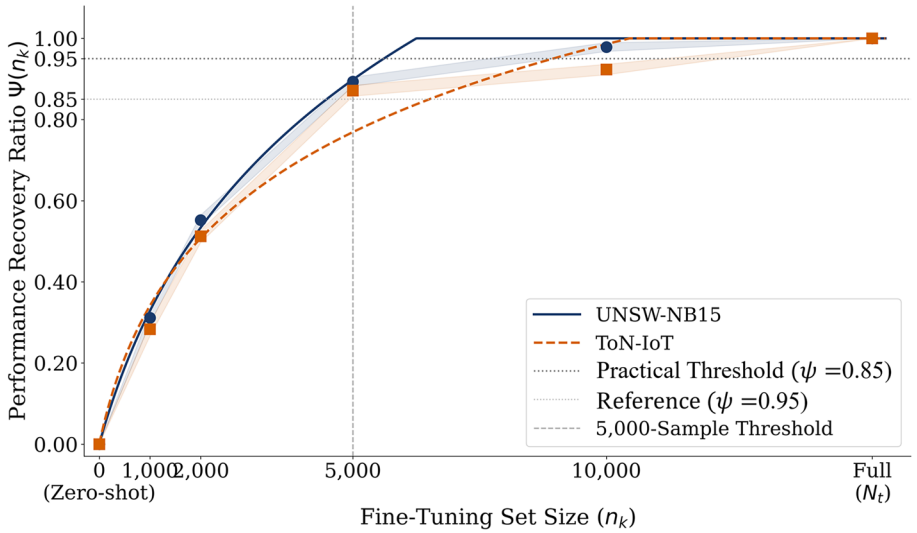


Fig. 8 Limited-label calibration: performance recovery ratio versus fine-tuning set size.

15 selected features are present in UNSW-NB15 and nine in ToN-IoT, with the absent features drawn predominantly from TCP-flag and protocol-specific categories, so the rate-based features dominating the top attribution ranks of Table 10 are largely preserved across all 3 datasets. UNSW-NB15 and ToN-IoT are general-purpose network and IoT intrusion datasets rather than medical-IoT traffic, and we use them here to measure transfer under feature-space and distribution shift. Zero-shot TAPX-Net already outperforms every baseline on both targets, leading BiLSTM by 3.78 points on UNSW-NB15 without any target exposure. The Transformer again records lower zero-shot recall than BiLSTM, repeating the shift-condition reversal of the zero-day results. Fine-tuning without MMD improves over zero-shot by 3.33 and 4.11 points on the two targets, and adding MMD alignment of Eq. (42) yields a further 4.56 and 4.78 points, the larger gain falling on the higher-shift ToN-IoT, consistent with the MMD penalty reducing discrepancy more where it is larger.

The calibration curve in Fig. 8 identifies 5,000 labeled target windows as the operating point on these two targets under the $\Psi \geq 0.85$ criterion: the recovery ratio of Eq. (44) reaches 0.8934 on UNSW-NB15 and 0.8712 on ToN-IoT at this size, both meeting the criterion. In comparison, increasing to 10,000 windows adds only 0.0855 and 0.0522 further recovery. The 1,000-window recovery of 0.3112 and 0.2834 marks a lower end of the tested range at which fine-tuning on these targets recovers less than a third of the achievable gain, and the widening UNSW-NB15 to ToN-IoT gap with fine-tuning size is consistent with the higher shift of ToN-IoT requiring proportionally more labelled data to achieve comparable domain alignment.

Ablation study

Table 15 reports each component's contribution through removal or replacement on CICIoMT2024, with zero-day recall included alongside batch metrics.

Table 15 Ablation Study — Component Contribution on CICIoMT2024.

Configuration	R_(FPR≤1%)	PR-AUC	F1	Zero-Day Recall	Latency (ms)	Memory (MB)
Full TAPX-Net	0.9541±0.0019	0.9612±0.0017	0.9574±0.0021	0.8963±0.0099	9.2	182.1
A1: No Patchification	0.9312±0.0031	0.9423±0.0028	0.9389±0.0034	0.8534±0.0134	8.7	164.3
A2: No SE Attention	0.9423±0.0024	0.9534±0.0021	0.9489±0.0027	0.8712±0.0118	9.1	178.6
A3: No Gated Residuals	0.9467±0.0022	0.9578±0.0019	0.9523±0.0024	0.8789±0.0112	9.0	179.8
A4: No Reconstruction Head	0.9512±0.0021	0.9601±0.0018	0.9556±0.0022	0.8217±0.0121	9.2	180.4
A5: Single BiLSTM Layer	0.9389±0.0028	0.9467±0.0025	0.9434±0.0029	0.8623±0.0127	8.4	156.7
A6: Standard Residuals	0.9478±0.0023	0.9589±0.0020	0.9534±0.0025	0.8834±0.0108	9.1	181.2
A7: No Positional Encoding	0.9356±0.0029	0.9445±0.0026	0.9412±0.0031	0.8589±0.0131	9.2	181.4
A8: MI-Only Feature Selection	0.9234±0.0037	0.9312±0.0033	0.9278±0.0039	0.8312±0.0148	9.3	183.7
A9: No Feature Selection (All 43 Features)	**0.9158±0.0043**	**0.9236±0.0039**	**0.9197±0.0045**	**0.8154±0.0162**	**11.7**	**205.8**

Removing the reconstruction head in A4 generates the largest single-component zero-day drop, 7.46 points, alongside the smallest batch R_(FPR≤1%) drop of 0.29 points, the sharpest cross-metric dissociation in Table 15, and proof that its contribution is remote to generalisation. Removing patchification in A1 generates the second-largest zero-day drop of 4.29 points with a more balanced batch drop of 2.29 points, indicating that the tempo.

The positional-encoding ablation A7 produces a larger batch drop than the SE ablation A2 but a smaller zero-day drop, a reversal signifying that positional encoding aids in-distribution localization while channel-attentive filtering aids cross-class generalization. The single-layer variant A5 shows that the second BiLSTM layer contributes more to in-distribution depth than to generalisation; replacing gated with standard residuals in A6 gives the smallest batch degradation, and the FS controls A8 and A9 record the lowest performance on every metric. The all-feature configuration A9, which applies no FS, provides the lowest values in the table on all 4 detection metrics together with the highest latency and memory; MI-only filtering in A8 improves every detection metric over A9 while reducing latency and memory, and the full MI+DL-BDA pipeline improves further over A8 on every metric. Memory varies non-monotonically among the architectural ablations, and all variants satisfy the constraint of Eq. (9), confirming the 182.1 MB footprint reflects a deliberate balance rather than a minimal configuration.

Statistical significance

Table 16 reports the Wilcoxon signed-rank statistic, Bonferroni-corrected p-value, and Cohen's d for TAPX-Net against each baseline, and Table 17 reports the McNemar results for transfer.

With 5 runs, the Wilcoxon test admits only two significant levels, 0.0312 at $W = 15$ and 0.0469 at $W = 14$. The Bonferroni-corrected threshold for the 28 comparisons is 0.0018, which no comparison clears, an expected consequence of the 5-run sample size rather than evidence of negligible differences; the Cohen's d values, computed independently of the p-value constraint, range from 1.178 to 7.895 and are uniformly large. The largest effects belong to the edge-Transformer comparisons, reflecting its quantisation deficit, and the smallest to BiLSTM, the architecturally closest baseline. Three comparisons at $W = 13$ on F1 or PR-AUC against intermediate baselines indicate TAPX-Net does not win on every run for those specific combinations, metrics less sensitive to the low-FPR regime targeted by the fitness of Eq. (14).

The McNemar test in Table 17 was computed with Edwards' continuity correction, using the asymptotic chi-square statistic $\frac{(|b-c|-1)^2}{b+c}$ with one degree of freedom, where b and c are the directional discordant counts. In every transfer comparison, b exceeds c. Because windows overlap under the stride configuration ($s = 16$, $T = 32$), the window-level decisions within a stream are not mutually independent, and the McNemar statistics are reported as

Table 16 Wilcoxon Signed-Rank Test — TAPX-Net *vs* Classifier Baselines.

Comparison	Metric	W	p-value	Cohen's d	Effect
vs. RF	R_(FPR≤1%)	15.0	0.0312	2.341	Large
	PR-AUC	15.0	0.0312	2.187	Large
	F1	15.0	0.0312	2.089	Large
	Zero-Day Recall	15.0	0.0312	2.512	Large
vs. LSTM	R_(FPR≤1%)	15.0	0.0312	1.834	Large
	PR-AUC	15.0	0.0312	1.712	Large
	F1	15.0	0.0312	1.689	Large
	Zero-Day Recall	15.0	0.0312	1.978	Large
*vs.*CNN-LSTM	R_(FPR≤1%)	15.0	0.0312	1.567	Large
	PR-AUC	14.0	0.0469	1.423	Large
	F1	13.0	0.0625	1.312	Large
	Zero-Day Recall	15.0	0.0312	1.712	Large
vs. BiLSTM	R_(FPR≤1%)	14.0	0.0469	1.234	Large
	PR-AUC	13.0	0.0625	1.178	Large
	F1	14.0	0.0469	1.212	Large
	Zero-Day Recall	15.0	0.0312	1.456	Large
vs. Transformer	R_(FPR≤1%)	15.0	0.0312	1.689	Large
	PR-AUC	14.0	0.0469	1.534	Large
	F1	13.0	0.0625	1.489	Large
	Zero-Day Recall	15.0	0.0312	1.823	Large
*vs.*GCN-Transformer	R_(FPR≤1%)	15.0	0.0312	2.541	Large
	PR-AUC	15.0	0.0312	3.680	Large
	F1	15.0	0.0312	2.015	Large
	Zero-Day Recall	15.0	0.0312	3.795	Large
vs. Lightweight Edge Transformer	R_(FPR≤1%)	15.0	0.0312	7.056	Large
	PR-AUC	15.0	0.0312	7.895	Large
	F1	15.0	0.0312	5.690	Large
	Zero-Day Recall	15.0	0.0312	5.146	Large

Table 17 McNemar Test — Cross-Dataset Transfer Agreement.

Comparison	Target	b (TAPX-Net correct, comparator wrong)	c (TAPX-Net wrong, comparator correct)	Discordant (b+c)	$\chi 2$	p-value
TAPX-Net (MMD) vs zero-shot	UNSW-NB15	1,907	1,505	3,412	47.13	<0.0001
TAPX-Net (MMD) vs No MMD	UNSW-NB15	1,226	961	2,187	31.87	<0.0001
TAPX-Net (MMD) vs BiLSTM	UNSW-NB15	3,784	3,005	6,789	89.16	<0.0001
TAPX-Net (MMD) vs zero-shot	ToN-IoT	1,663	1,271	2,934	52.11	<0.0001
TAPX-Net (MMD) vs No MMD	ToN-IoT	1,413	1,099	2,512	39.00	<0.0001
TAPX-Net (MMD) vs BiLSTM	ToN-IoT	3,187	2,447	5,634	96.93	<0.0001

Table 18 Loss-Weighting Sensitivity on CICIoMT2024.

λ_rec	R_(FPR≤1%)	PR-AUC	F1	Zero-Day Recall
0.00	0.9512±0.0021	0.9601±0.0018	0.9556±0.0022	0.8217±0.0121
0.01	0.9520±0.0022	0.9604±0.0019	0.9560±0.0023	0.8486±0.0118
0.05	0.9534±0.0020	0.9609±0.0018	0.9568±0.0021	0.8827±0.0106
0.10	0.9541±0.0019	0.9612±0.0017	0.9574±0.0021	0.8963±0.0099
0.30	0.9535±0.0021	0.9608±0.0019	0.9569±0.0022	0.8891±0.0104
0.50	0.9526±0.0023	0.9600±0.0020	0.9559±0.0024	0.8754±0.0112
1.00	0.9498±0.0027	0.9571±0.0024	0.9527±0.0028	0.8428±0.0131

Table 19 Patch-Count Sensitivity on CICIoMT2024.

Patch Count P	RFPR≤1%	PR-AUC	F1	Zero-Day Recall	Latency (*ms*)	Memory (MB)
2	0.9492 ± 0.0024	0.9568 ± 0.0022	0.9526 ± 0.0029	0.8784 ± 0.0121	8.6	177.4
4	0.9541 ± 0.0019	0.9612 ± 0.0017	0.9574 ± 0.0020	0.8963 ± 0.0099	9.2	182.1
8	0.9517 ± 0.0022	0.9589 ± 0.0019	0.9548 ± 0.0028	0.8871 ± 0.0108	10.4	188.7
16	0.9465 ± 0.0028	0.9540 ± 0.0025	0.9503 ± 0.0029	0.8696 ± 0.0137	12.8	201.6

a descriptive summary of the paired per-window decision differences rather than as an exact independent-sample hypothesis test.

Loss-weighting sensitivity

Table 18 reports the sensitivity of detection performance to the reconstruction weight λ_{rec} of Eq. (39), swept over seven values on CICIoMT2024 with all other hyperparameters fixed.

The setting $\lambda_{\text{rec}} = 0$ matches the reconstruction ablation A4 of Table 15. All 4 metrics peak at $\lambda_{\text{rec}} = 0.10$ and decline on both sides, with zero-day recall varying by 7.46 points across the sweep against 0.43 points for R_(FPR≤1%), and zero-day standard deviation also lowest at 0.10. Wilcoxon tests comparing 0.10 against each other value favor TAPX-Net on every run for all four metrics, and Cohen's d exceeds 2.0 for every batch metric and 14 for zero-day recall; under Bonferroni correction for 24 comparisons, none clears 0.0021, the same small-sample constraint (Section "Statistical significance"). This study retains 0.10 (Table 2).

Patch-count sensitivity

Table 19 reports the sensitivity of detection performance to the patch count P of Eq. (15), evaluated over the admissible values $P \in \{2,4,8,16\}$ defined in Section "Patchification and positional embedding", with all other hyperparameters fixed and $T = 32$.

All 4 detection metrics peak at $P = 4$ and decline on both sides. At $P = 2$, each patch spans 16 observations, and $R_{\text{FPR}\leq 1\%}$ is 0.9492 ± 0.0024 with zero-day recall 0.8784 ± 0.0121. The selected configuration $P = 4$, corresponding to eight observations per patch, reaches 0.9541 ± 0.0019 at FPR $\leq 1\%$ and 0.8963 ± 0.0099 zero-day recall. Increasing the patch count to $P = 8$ reduces degradation, and $P = 16$, corresponding to two observations per patch, reduces zero-day recall to 0.8696 ± 0.0137. Latency and memory increase monotonically with the patch count, from 8.6 ms and 177.4 MB at $P = 2$ to 12.8 ms and 201.6 MB at $P = 16$, so the $P = 16$ configuration also exceeds the 10 ms deployment bound. The value $P = 4$ is retained in Table 2.

Probabilistic calibration

Table 20 reports the ECE across 10 bins and 5 runs, computed per Section "Hardware and software". TAPX-Net's predictions are well calibrated, with ECE from 0.0066 to 0.0093 and a mean of 0.0080±0.0011, well below the 0.05 to 0.10 range commonly treated as acceptable for deep classifiers. Calibration is weakest in the extreme bins,

Table 20 Probabilistic Calibration — TAPX-Net on CICIoMT2024.

Run	ECE
1	0.0079
2	0.0066
3	0.0093
4	0.0074
5	0.0087
Mean ± SD	0.0080 ± 0.0011

where the highest-probability bin underestimates the observed positive rate by the largest per-bin margin and contributes most of the total error, while the intermediate bins agree closely. The pattern is consistent across runs, with the model best calibrated in the high-probability bins that contain most test windows and comparatively less precise, though still small in absolute terms, in the sparsely populated middle bins.

Conclusion and future work

This paper addressed streaming ID in medical IoT traffic as a window-local temporal classification problem bounded by latency, interpretability, and cross-domain generalisation requirements. TAPX-Net combines temporal patch decomposition, channel-attentive convolutional filtering, and stacked bidirectional recurrence with a validated attribution module and a limited-label adaptation stage; its evaluation isolates each component's contribution rather than reporting aggregate performance alone.

The results support three findings. First, patch decomposition with channel-attentive filtering and the reconstruction head together produce detection performance exceeding seven baselines on the CICIoMT2024 benchmark while meeting the bounded per-window latency measured on the tested CPU configuration, and the reconstruction head contributes its improvement specifically in the zero-day regime, where it raises recall by 7.46 percentage points against a variant trained without it. Second, the attribution module produces explanations whose IG component tracks a causal input-level ablation more closely than attention weights across attack classes, and the per-class feature profiles align with domain-known attack signatures, with attention admitted as a supplementary signal only where it agrees with the causal ablation. Third, limited-label adaptation with distribution alignment transfers the model to two general-purpose external network domains and recovers most of the achievable performance gain at 5,000 labelled target windows, with the alignment penalty contributing a larger gain on the higher-shift target.

Several limitations bound these conclusions. The evaluation uses processed cybersecurity traffic benchmarks rather than live clinical deployments, and the reported latency, measured under CPU-only inference on a single node, should be read as a lower bound rather than a guarantee on more constrained edge hardware. The latency and memory measurements were obtained on an Intel Core i7-12700H CPU and were not measured on ARM-based or other resource-constrained edge processors, so the reported figures are not a substitute for on-device measurement. The model is window-local and carries no state across window boundaries, so temporal dependencies spanning longer than one window are not modelled; within a window, the bidirectional layers still provide each patch with the context of all other patches, and consecutive windows overlap under the stride setting of Table 2, so intra-window and short-range cross-window structure remains available even though no recurrent state is broadcast. The domain-adaptation stage aligns the marginal representation distributions of source and target through MMD and adjusts the model with labeled target windows. However, it does not explicitly align the class-conditional distributions, so it does not directly address residual conditional shift between domains. The attention agreement analysis admits attention as an attribution component for 8 of 10 attack classes, and the two MQTT classes rely on IG alone, indicating that attention accuracy is not uniform across attack families. The statistical comparisons rest on 5 independent runs, a sample size at which Bonferroni-corrected significance is

not attainable despite uniformly large effect sizes, and the cross-domain results cover two target datasets rather than a clinical network.

Future work follows from these limits. Evaluation on live clinical traffic and on constrained edge hardware, including ARM-based edge gateways under on-device measurement, would establish whether the latency and memory measurements hold under deployment conditions. Modelling temporal dependencies beyond a single window, without reintroducing the bounded-latency cost that motivated the window-local design, would extend the method to attacks whose signature spans multiple windows. Aligning the class-conditional distributions during transfer, for example through conditional or joint distribution matching, would address the residual conditional shift that marginal alignment leaves. Improving attention accuracy on protocol-specific attack families such as MQTT, and widening the cross-domain evaluation to additional network environments, would strengthen the generalisation claims.

Acknowledgements Not applicable.

Author contributions Conceptualization, Methodology, Software, Validation, Formal analysis, Investigation, Resources, Data Curation, Writing—Original Draft, Writing—Review & Editing, Visualization : P. Saranya , S. Sudhakar.

Funding Not applicable.

Data availability CICIoMT2024 / CIC IoMT Dataset 2024: https://www.unb.ca/cic/datasets/iomt-dataset-2024.html; UNSW-NB15 Dataset : https://research.unsw.edu.au/projects/unsw-nb15-dataset; ToN-IoT Dataset : https://research.unsw.edu.au/projects/toniot-datasets. Code Availability: The source code and implementation of the proposed explainable streaming intrusion detection framework are publicly available on GitHub: https://github.com/Saranya-14-10/Explainable-Streaming-IDS. The repository includes the TAPX-Net implementation, an executable Jupyter/Google Colab notebook, a Python execution script, dependency specifications, and reproducibility documentation. The associated research artifacts and dataset information are additionally archived through Zenodo: https://zenodo.org/records/22657257. Public demonstration datasets are accessed through their respective sources, while the repository does not redistribute restricted datasets.

Code availability The source code and implementation of the proposed explainable streaming intrusion detection framework are publicly available on GitHub: https://github.com/Saranya-14-10/Explainable-Streaming-IDS. The repository includes the TAPX-Net implementation, an executable Jupyter/Google Colab notebook, a Python execution script, dependency specifications, and reproducibility documentation. The associated research artifacts and dataset information are additionally archived through **Zenodo:**https://zenodo.org/records/22657257. Public demonstration datasets are accessed through their respective sources, while the repository does not redistribute restricted datasets.

Declarations

Competing interests Not applicable.

Open Access This article is licensed under a Creative Commons Attribution-NonCommercial-NoDerivatives 4.0 International License, which permits any non-commercial use, sharing, distribution and reproduction in any medium or format, as long as you give appropriate credit to the original author(s) and the source, provide a link to the Creative Commons licence, and indicate if you modified the licensed material. You do not have permission under this licence to share adapted material derived from this article or parts of it. The images or other third party material in this article are included in the article's Creative Commons licence, unless indicated otherwise in a credit line to the material. If material is not included in the article's Creative Commons licence and your intended use is not permitted by statutory regulation or exceeds the permitted use, you will need to obtain permission directly from the copyright holder. To view a copy of this licence, visit http://creativecommons.org/licenses/by-nc-nd/4.0/.

References

1. Hassan, S. R., Tanveer, M. U., Prajapat, S. & Shabaz, M. A comprehensive survey on intrusion detection in Internet of Medical Things: Datasets, federated learning, blockchain, and future research directions. *ICT Express*https://doi.org/10.1016/j.icte.2025.11.005 (2025).

2. Doménech, J., León, O., Siddiqui, M. S. & Pegueroles, J. Evaluating and enhancing intrusion detection systems in IoMT: The importance of domain-specific datasets. *Internet Things***32**, 101631 (2025).
3. Wang, T., Xu, Y. & Tang, Z. Toward fast network intrusion detection for web services: Partial-flow feature extraction and dataset construction. *Int. J. Web Inf. Syst.***21**(1), 77–95 (2025).
4. Puccetti, T. & Ceccarelli, A. On detection latencies of network intrusion detectors: Discussion and application. *Empir. Softw. Eng.***31**, 23 (2026).
5. Hosain, Y. & Çakmak, M. XAI-XGBoost: An innovative explainable intrusion detection approach for securing Internet of Medical Things systems. *Sci. Rep.***15**, 22278 (2025).
6. Georgiades, M. & Hussain, F. An explainable AI approach for interpretable cross-layer intrusion detection in Internet of Medical Things. *Electronics***14**(16), 3218 (2025).
7. Šimić, I., Veas, E. & Sabol, V. A comprehensive analysis of perturbation methods in explainable AI feature attribution validation for neural time series classifiers. *Sci. Rep.***15**, 26607 (2025).
8. Vardas, E. P., Marketou, M. & Vardas, P. E. Medicine, healthcare and the AI act: Gaps, challenges and future implications. *Eur. Heart J. Digit. Health***6**(4), 833–839 (2025).
9. Mahbub, M., Riasat, M. T., Hamid, T., Sutradhar, S. C. & Khan, M. S. A. A minimalistic yet effective domain adaptation strategy for IoMT network intrusion detection. *Discover Internet of Things***6**, 28 (2026).
10. Palaniappan, S. & Sengan, S. Hybrid feature selection for IoMT-based intrusion detection system for integrating mutual information filtering with deep learning-based accelerated metaheuristic optimization. *Sci. Rep.*https://doi.org/10.1038/s41598-026-47264-5 (2026).
11. Akar, G., Sahmoud, S., Onat, M., Cavusoglu, Ü. & Malondo, E. L2D2: A novel LSTM model for multi-class intrusion detection systems in the era of IoMT. *IEEE Access***13**, 7002–7013 (2025).
12. Berguiga, A., Harchay, A. & Massaoudi, A. HIDS-IoMT: A deep learning-based intelligent intrusion detection system for the Internet of Medical Things. *IEEE Access***13**, 32863–32882 (2025).
13. Mezina, A., Nurmi, J. & Ometov, A. Novel hybrid UNet++ and LSTM model for enhanced attack detection and classification in IoMT traffic. *IEEE Access***13**, 57589–57603 (2025).
14. Shaikh, J. A., Wang, C., Sima, M. W. U., Arshad, M., Owais, M., Hassan, D. S. M., Alkanhel, R., & Muthanna, M. S. A. (2025a). A deep reinforcement learning-based robust intrusion detection system for securing IoMT healthcare networks—Frontiers in Medicine, 12, 1524286.
15. Shaikh, J. A., Wang, C., Saifullah, Sima, M. W. U., Arshad, M., & Rathore, W. U. A. (2025b). Memory feedback transformer-based intrusion detection system for IoMT healthcare networks—Internet of Things, 32, 101597.
16. Li, B., Li, J. & Jia, M. ADFCNN-BiLSTM: A deep neural network based on attention and deformable convolution for network intrusion detection. *Sensors***25**(5), 1382 (2025).
17. Alashjaee, A. M. Deep learning for network security: An Attention-CNN-LSTM model for accurate intrusion detection. *Sci. Rep.***15**, 21856 (2025).
18. Ghosh, K. P., Hasan, M., Robin, M. T. I., Hossain, M. A. & Islam, M. S. A novel deep learning framework with temporal attention convolutional networks for intrusion detection in IoT and IIoT networks. *Sci. Rep.***15**, 44624 (2025).
19. Alitbi, Z. K., Hosseini Seno, S. A., Ghaemi Bafghi, A. & Zabihzadeh, D. A generalized and real-time network intrusion detection system through incremental feature encoding and similarity embedding learning. *Sensors***25**(16), 4961 (2025).
20. Alabbadi, A. & Bajaber, F. An intrusion detection system over the IoT data streams using explainable artificial intelligence. *Sensors***25**(3), 847 (2025).
21. Lipsa, S., Dash, R. K. & Ivković, N. An interpretable dimensional reduction technique with an explainable model for detecting attacks in Internet of Medical Things devices. *Sci. Rep.***15**, 8718. https://doi.org/10.1038/s41598-025-93404-8 (2025).
22. Hafid, A., Rahouti, M. & Aledhari, M. Optimizing intrusion detection in IoMT networks through interpretable and cost-aware machine learning. *Mathematics***13**(10), 1574 (2025).
23. Alsharaiah, M. A. et al. An explainable AI-driven transformer model for spoofing attack detection in Internet of Medical Things networks. *Discover Applied Sciences***7**, 488 (2025).
24. Bin Hulayyil, S., Li, S. & Saxena, N. Explainable AI-based intrusion detection in IoT systems. *Internet Things***31**, 101589 (2025).
25. Krishnan, D., Singh, S. & Sugumaran, V. Explainable AI for zero-day attack detection in IoT networks using attention fusion model. *Discover Internet of Things***5**, 83 (2025).
26. Alabbadi, A. & Bajaber, F. X-FuseRLSTM: A cross-domain explainable intrusion detection framework in IoT using attention-guided dual-path feature fusion and residual LSTM. *Sensors***25**(12), 3693 (2025).
27. Kalakoti, R., Nõmm, S. & Bahsi, H. Federated learning of explainable AI for deep learning-based intrusion detection in IoT networks. *Comput. Networks***270**, 111479 (2025).
28. Fahim-Ul-Islam, M., Chakrabarty, A., Alam, M. G. R. & Maidin, S. S. A resource-efficient federated learning framework for intrusion detection in IoMT networks. *IEEE Trans. Consum. Electron.***71**(2), 4508–4521 (2025).
29. Zhang, J., Li, Y. & Zhang, L. Heterogeneous network intrusion detection via domain adaptation in IoT environment. *Intern. Technol. Lett.***8**(1), e531 (2025).

30. Jin, H. Cross-protocol domain gap in Internet of Things intrusion and anomaly detection: An empirical Internet Protocol-to-Bluetooth Low Energy study of domain-adversarial training. *Sensors***26**(4), 1184 (2026).
31. Jain, S. & Wallace, B. C. Attention is not explanation. *Proceed. NAACL-HLT***2019**, 3543–3556 (2019).
32. Sundararajan, M., Taly, A., & Yan, Q. (2017). Axiomatic attribution for deep networks. Proceedings of the 34th International Conference on Machine Learning, 70, 3319–3328.
33. Dadkhah, S. et al. CICIoMT2024: A benchmark dataset for multi-protocol security assessment in IoMT. *Internet Things***28**, 101351. https://doi.org/10.1016/j.iot.2024.101351 (2024).
34. Moustafa, N., & Slay, J. (2015). UNSW-NB15: A comprehensive data set for network intrusion detection systems. Military Communications and Information Systems Conference (MilCIS), 1–6.
35. Alsaedi, A., Moustafa, N., Tari, Z., Mahmood, A. & Anwar, A. TON_IoT telemetry dataset: A new generation dataset of IoT and IIoT for data-driven intrusion detection systems. *IEEE Access***8**, 165130–165150 (2020).
36. Khan, M. Z., Sabur, A. & Ghandorh, H. A novel Internet of Medical Things hybrid model for cybersecurity anomaly detection. *Sensors***25**(20), 6501. https://doi.org/10.3390/s25206501 (2025).
37. Shi, Q., Yang, Z. & Shao, S. TransEdge: A lightweight transformer-based anomaly detection framework for edge devices. *J. Intell. Inf. Syst.***64**, 1747–1775. https://doi.org/10.1007/s10844-026-01043-w (2026).
38. Lundberg, S. M. & Lee, S.-I. A unified approach to interpreting model predictions. *Adv. Neural Inf. Process. Syst.***30**, 4765–4774 (2017).
39. Cohen, J. *Statistical Power Analysis for the Behavioral Sciences* 2nd ed. (Lawrence Erlbaum Associates, 1988).
40. Zheng, C. et al. Answering subset query over multi-attribute data streams using Hyper-USS. *IEEE Trans. Knowl. Data Eng.***37**(12), 6916–6933. https://doi.org/10.1109/TKDE.2025.3611502 (2025).
41. Wang, M., Zhou, D. & Chen, M. Hybrid variable monitoring: An unsupervised process monitoring framework with binary and continuous variables. *Automatica***147**, Article 110670. https://doi.org/10.1016/j.automatica.2022.110670 (2023).
42. Zhang, W. et al. MODIFy: A multi-modal anomaly diagnosis framework with diffusion-enhanced adaptive fusion in microservices. *J. Syst. Softw.***237**, Article 112844. https://doi.org/10.1016/j.jss.2026.112844 (2026).
43. Qin, Y. et al. Fault localization from the semantic code search perspective. *ACM Trans. Softw. Eng. Methodol.*https://doi.org/10.1145/3757915 (2026).
44. Guo, C., Zhong, G., Xiao, Z. & Wang, D. Detecting faulty substructures in networks. *IEEE Trans. Comput.***75**(5), 1937–1948. https://doi.org/10.1109/TC.2026.3661204 (2026).
45. Jin, J., Wu, M., Ouyang, A., Li, K. & Chen, C. A novel dynamic Hill cipher and its applications on medical IoT. *IEEE Internet Things J.***12**(10), 14297–14308. https://doi.org/10.1109/JIOT.2025.3525623 (2025).

Publisher's note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.

Authors and Affiliations

P. Saranya[1] · S. Sudhakar[2]

✉ P. Saranya
saranya_p1@outlook.com

✉ S. Sudhakar
dr.s.sudhakar@gmail.com

[1] Department of Computer Science and Engineering, AIML, K S Rangasamy College of Technology, Tiruchengode, Tamil Nadu 637215, India

[2] Department of Computer Science and Engineering, Anna University, Chennai, Tamil Nadu 600025, India